

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Machine Learning for Species Diversification Dynamics

Tesi di Laurea Magistrale in
High Performance Computing Engineering -
Ingegneria del Calcolo ad Alte Prestazioni

Author: **Mattia Colbertaldo**

Student ID: 224504
Advisor: Prof. Cristina Silvano
Co-advisors: Prof. Ernst Wit, Dr. Francisco Richter Mendoza (Università della Svizzera Italiana)
Academic Year: 2023-24

Abstract

This study explores the application of deep learning techniques for inferring parameters of diversification models from extant phylogenies, providing an alternative to the slow and computationally expensive Maximum Likelihood Estimation (MLE) methods, which often lack analytical solutions. The research evaluates Convolutional Neural Networks (CNNs), Linear models, Feed Forward Neural Networks (FFNNs), Gated Recurrent Units (GRUs), and Long Short-Term Memory (LSTM) Networks. Despite computational constraints, promising results were achieved. The analysis focused on two types of encoding: CDV and Summary Statistics. A significant contribution of this work is the introduction of a Random Forest Classifier for parameter inference, which exhibited superior predictive accuracy and faster training times compared to other deep learning models. The study also highlighted the limitations of previous research due to overly stringent constraints. A novel tool capable of inferring the mathematical model of a given phylogeny was developed, dramatically reducing computational time from minutes to milliseconds and achieving an impressive test accuracy of 88%. This advancement enables efficient and accurate inference of diversification models, even in scenarios where MLE was infeasible.

The research leverages the diversitree package for simulating phylogenetic trees under various models, selecting it for its comprehensive and customizable functions. This study represents a significant step towards developing sophisticated and scalable analytical tools for evolutionary biology.

Keywords: machine learning, phylogenetics, biology, deep learning, evolution

Abstract in lingua italiana

Questo studio esplora l'applicazione delle tecniche di deep learning per dedurre i parametri dei modelli di diversificazione dalle filogenie (gli alberi genealogici che mostrano le relazioni evolutive tra specie), fornendo un'alternativa ai metodi di Massima Verosimiglianza (MLE), che sono lenti e computazionalmente costosi e spesso privi di soluzioni analitiche. La ricerca valuta Convolutional Neural Networks (CNNs), modelli Lineari, Feed Forward Neural Networks (FFNNs), Gated Recurrent Units (GRUs) e Long Short-Term Memory (LSTM) Networks. Nonostante i vincoli computazionali, sono stati ottenuti risultati promettenti. L'analisi si è concentrata su due tipi di codifica: CDV e Summary Statistics. Un contributo significativo di questo lavoro è l'introduzione di un Random Forest Classifier per l'inferenza dei parametri, che ha mostrato una precisione predittiva superiore e tempi di addestramento più rapidi rispetto agli altri modelli di deep learning. Lo studio ha anche evidenziato i limiti delle ricerche precedenti dovuti a vincoli eccessivamente rigidi. È stato sviluppato un nuovo strumento capace di inferire il modello matematico di una data filogenia, riducendo drasticamente il tempo computazionale da minuti a millisecondi e raggiungendo un'accuratezza dell' 88%. Questo progresso consente un'inferenza efficiente e accurata dei modelli di diversificazione, anche in scenari in cui l'MLE era impraticabile.

La ricerca sfrutta il pacchetto diversitree per simulare alberi filogenetici sotto vari modelli, selezionandolo per le sue funzioni complete e personalizzabili. Questo studio rappresenta un passo significativo verso lo sviluppo di strumenti analitici sofisticati e scalabili per la biologia evolutiva.

Parole chiave: machine learning, filogenetica, biologia, deep learning, evoluzione

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

Introduction **1**

1 Models **3**
1.1 Constant Rate Birth-Death . . . 5
1.2 BiSSE . . . 6
1.3 MuSSE . . . 7
1.4 ClaSSE . . . 8
1.5 GeoSSE . . . 10
1.6 QuaSSE . . . 11
1.7 BiSSEness . . . 12
1.8 Overview of models . . . 14

2 Simulations **15**
2.1 Real data analysis for parameters range retrieval . . . 15
2.2 Generating trees . . . 17
2.3 Other packages . . . 18
2.3.1 `ape` package: . . . 18
2.3.2 `geiger` package: . . . 20
2.3.3 `TreeSim` package: . . . 20
2.3.4 `TreeSimGM` package: . . . 21
2.3.5 `phytools` package: . . . 21

3 Extending Deep Learning Techniques to Complex Phylogenetic Models **25**
3.1 Addressing Ill-Posedness of the Problem . . . 27

3.2 Training Results 28
3.3 Conclusions 35

4 Impact of parameter constraints and data manipulation on the results 37
4.1 Available configurations 38
4.2 Parameter Constraints 39
4.3 Results 39
4.4 Artificial inflation of the results 40
4.5 Removing Constraints 46

5 Model Predictor 49
5.1 Random Forest Classifier 49
5.2 Multi-Layer Perceptron Classifier 50
5.3 Deep Neural Network 50
5.4 Convolutional Neural Network 52
5.5 Conclusions 53

6 Conclusions and future developments 57

Bibliography 59

Appendix 61
.1 Examples 61
.2 Models 68

List of Figures 73

List of Tables 77

List of Symbols 79

Acknowledgements 81

Introduction

Phylogenetic inference, a critical task in evolutionary biology, aims to reconstruct the evolutionary relationships among species based on various types of data, including genetic, morphological, and ecological information. Traditional methods such as Maximum Likelihood Estimation (MLE) and Bayesian Inference have been widely employed due to their statistical robustness and comprehensive frameworks. However, these approaches often struggle with scalability and computational efficiency, particularly as the complexity and volume of biological data increase.

In recent years, the application of deep learning techniques to phylogenetic inference has emerged as a promising avenue. Deep learning, characterized by neural networks with multiple layers, excels in pattern recognition and predictive modeling across various domains. Its potential to handle large-scale and complex datasets makes it an attractive alternative for phylogenetic studies.

This thesis aims to extend previous studies by applying deep learning to more sophisticated phylogenetic models, specifically the MuSSE (Multiple State Speciation and Extinction), GeoSSE (Geographical State Speciation and Extinction), and BiSSEness (Binary-State Speciation and Extinction with node enhancements) models. These models introduce additional complexities such as multiple character states, geographical considerations, and state shifts during speciation events, which are not adequately addressed by simpler models.

One significant challenge addressed in this research is the ill-posed nature of the parameter estimation problem in phylogenetics. This problem arises when different combinations of parameters can produce similar trees, making it difficult for models to learn accurate relationships. This issue is particularly pronounced in more complex models like MuSSE, where the increased number of parameters make deriving unique solutions from the given data more challenging.

Furthermore, this research seeks to determine the most suitable neural network architecture and data representation for various phylogenetic inference tasks. The evaluation includes different deep learning architectures—such as Deep Neural Networks (DNNs),

Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) such as Gated Recurrent Unit (GRU) and Long Short Term Memory (LSTM); and phylogenetic data representations, including summary statistics, Lineage Through Time (LTT) plots, phylogeny encoding, and phylogeny graphs.

In addition, a novel model predictor was created to predict the underlying phylogenetic model that generated a given tree. Various classification models, including Random Forest Classifiers, Multi-Layer Perceptron Classifiers, Deep Neural Networks, and Convolutional Neural Networks, were trained on trees generated by different models. The trees were encoded using both CDV (Cumulative Distribution Vector) and SS (Summary Statistics) encodings. The classification models were evaluated on their accuracy, and the best-performing model was selected.

By systematically comparing these combinations, the study aims to identify the optimal configurations for accurately inferring parameters of complex phylogenetic models. Preliminary findings suggest that deep learning methods can achieve comparable or superior accuracy to traditional MLE methods when appropriately tailored. For instance, Neural Networks have shown promising results for the Constant Rate Birth-Death (CRBD) and BiSSE (Binary State Speciation and Extinction) models, while more intricate models like BiSSEness require incorporating detailed topological and state information for effective inference.

Ultimately, this thesis underscores the potential of deep learning to revolutionize phylogenetic inference, offering enhanced efficiency and comparable accuracy. The insights gained from this research will contribute to the broader application of machine learning in evolutionary biology, paving the way for more sophisticated and scalable analytical tools.

This thesis has been developed at the end of the Double Master Degree in High Performance Computing Engineering at Politecnico di Milano and Università della Svizzera Italiana as part of the EUMaster4HPC European Program.

1 | Models

Diversification models are analytical frameworks used in evolutionary biology to study the processes that drive the emergence and extinction of species over time. These models help scientists understand how different factors, such as physical traits, behaviors, and environmental changes, influence the rates of speciation (the formation of new species) and extinction (the loss of species). By analyzing phylogenetic trees—diagrams that show the evolutionary relationships among species—diversification models can reveal patterns and trends in biodiversity. Examples of diversification models include the CRBD (Cladogenetic Birth-Death), BiSSE (Binary State Speciation and Extinction), and MuSSE (Multiple State Speciation and Extinction) models. These models are essential for reconstructing evolutionary histories, predicting future biodiversity changes, and informing conservation strategies by identifying key traits and environmental factors that impact species survival and diversification.

For instance, woodpeckers with diverse drumming behaviors (Fig. 1.1) exemplify a Multiple State Speciation and Extinction model (MuSSE). We can split the phylogeny in three states: regular drummers, occasional drummers and non-drummers. Their drumming intricacy and frequency correlate with speciation and extinction probabilities. Complex drumming aids territory defense and mate attraction, reducing gene flow and fostering speciation. Conversely, species with simpler drumming might face higher extinction risks, particularly in densely populated areas. This interplay showcases how macro behavioral or physical traits, called states, impact evolutionary dynamics, underscoring the significance of diversification models in understanding biodiversity trends.

This thesis will mainly focus on the following models:

- CRBD: Constant Rate Birth - Death.
- BiSSE: Binary trait speciation and extinction [1], and extensions for incompletely resolved phylogenies [2].

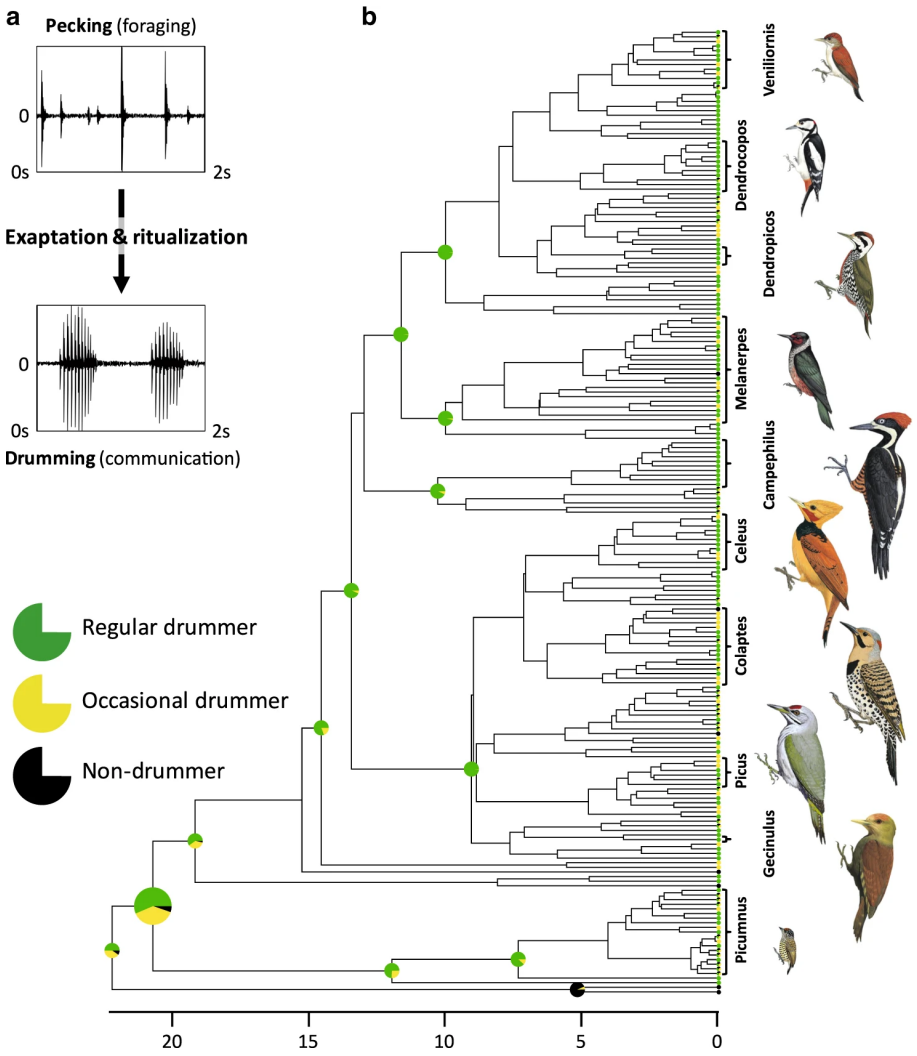


Figure 1.1: Drumming, among woodpeckers species, can be seen as a state differentiation, as it can lead to different speciation and extinction rates. From the figure, it can be seen that non-drummers have a lower speciation and an higer extinction rates, compared to drummers. [12]

- MuSSE: Multiple State Speciation and Extinction.
- QuaSSE: Quantitative State Speciation and Extinction.
- GeoSSE: Geographic State Speciation and Extinction [4].
- BiSSEness: BiSSE-Node Enhanced State Shift [5].
- ClaSSE: Cladogenetic State Speciation and Extinction.

1.1. Constant Rate Birth-Death

Simulation function: `tree.bd()`

Parameters:

- λ: Speciation rate per unit of time.
- μ: Extinction rate per unit of time.

Description: It models pure birth-death processes without any character state transitions. At each step of the Gillespie algorithm, the waiting time till the next event (speciation or extinction) is sampled from an exponential distribution with rate calculated as the sum of all propensity functions for each type of event. Then, the actual event is chosen based on its relative proportion to the total rate.

- Probability of increasing by one species (speciation):

$$P(N(t+\Delta t) = n+1 \mid N(t) = n) = n\lambda\Delta t + o(\Delta t)$$

- Probability of decreasing by one species (extinction):

$$P(N(t+\Delta t) = n-1 \mid N(t) = n) = n\mu\Delta t + o(\Delta t)$$

- Probability of no change:

$$P(N(t+\Delta t) = n \mid N(t) = n) = 1 - n(\lambda+\mu)\Delta t + o(\Delta t)$$

- Differential equations for survival probabilities $S(t)$ and speciation $B(t)$:

$$\frac{dS(t)}{dt} = -(\lambda+\mu)S(t)$$

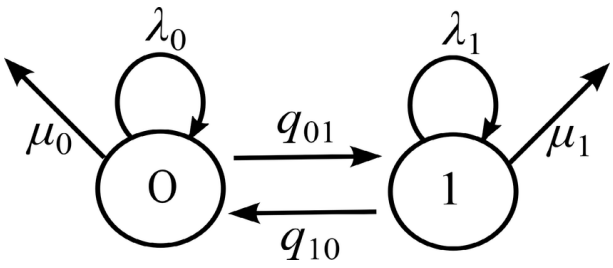


Figure 1.2: BiSSE model: λ represents the speciation rate for each state, while μ the extinction rate. q is the probability of transitioning from one state to another.

$$\frac{dB(t)}{dt} = \lambda S(t)$$

The $o(\Delta t)$ terms represent small higher-order contributions to the probability changes which are negligible as Δt approaches zero.

1.2. BiSSE

The BiSSE (Binary State Speciation and Extinction) model combines the features of the constant-rates birth-death model with the two-state Markov model. The BiSSE model assumes constant rates of speciation (λ_1, λ_2), extinction (μ_1, μ_2), and transition (q_{12}, q_{21}) between the two character states.

Simulation function: `tree.bisse()`

Parameters:

- Character-specific rates of speciation per unit of time.

$$\lambda_1 \quad \text{(for state 1)}$$

$$\lambda_2 \quad \text{(for state 2)}$$

- Character-specific rates of extinction per unit of time.

$$\mu_1 \quad \text{(for state 1)}$$

$$\mu_2 \quad \text{(for state 2)}$$

- Transition rates between character states per unit of time.

$$q_{12} \quad \text{(from state 1 to state 2)}$$

$$q_{21} \quad \text{(from state 2 to state 1)}$$

Differential equations for the survival probabilities $P_i(t)$ for each state i:

$$\frac{dP_1(t)}{dt} = -(\lambda_1 + \mu_1 + q_{12})P_1(t) + q_{21}P_2(t) + 2\lambda_1 P_1(t)P_1(t)$$

$$\frac{dP_2(t)}{dt} = -(\lambda_2 + \mu_2 + q_{21})P_2(t) + q_{12}P_1(t) + 2\lambda_2 P_2(t)P_2(t)$$

1.3. MuSSE

The MuSSE model generalizes BiSSE to accommodate characters with more than two states. Following Pagel (1994) [6], this can also allow for multiple characters, each of which might be binary, by recoding the states.

Simulation function: `tree.musse()`

Parameters:

- $\lambda_1, \ldots, \lambda_K$: State-specific rates of speciation per unit of time.

$$\lambda_i \quad \text{(for state } i,\ i = 1, 2, \ldots, K)$$

- $\mu_1, \ldots, \mu_K$: State-specific rates of extinction per unit of time.

$$\mu_i \quad \text{(for state } i,\ i = 1, 2, \ldots, K)$$

- q_{ij}: Probability of transitioning between character states per unit of time.

$$q_{ij} \quad \text{(from state } i \text{ to state } j,\ i, j = 1, 2, \ldots, K,\ i \neq j)$$

Differential equations for the survival probabilities $P_i(t)$ for each state i:

$$\frac{dP_i(t)}{dt} = -(\lambda_i + \mu_i + \sum_{j \neq i} q_{ij})P_i(t) + \sum_{j \neq i} q_{ji}P_j(t) + 2\lambda_i P_i(t)P_i(t)$$

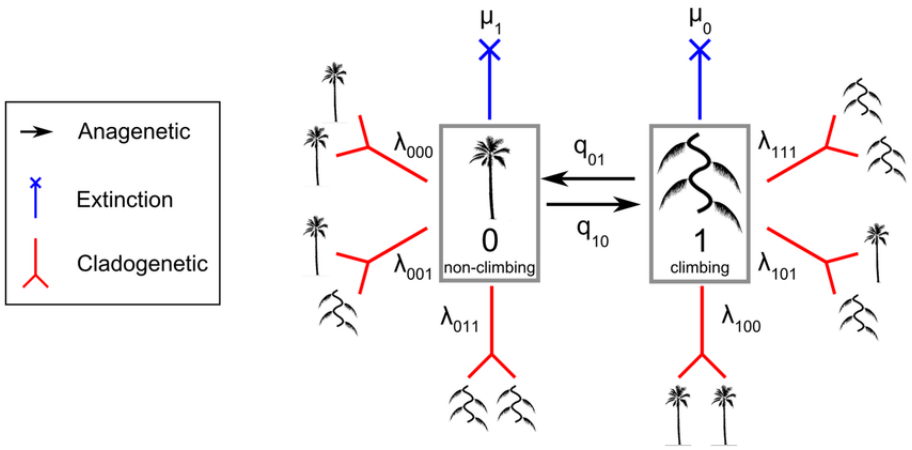


Figure 1.3: Two-states ClaSSE model: global diversification of a tropical plant growth form [10].

1.4. ClaSSE

Cladogenetic State Speciation and Extinction. Classe model considers as speciation rates every possible combination of speciation independently, in a cladogenetic environment - species can change state only during speciation and not along lineages (anagenesis). Anagenesis is included in the model thanks to the transition rates. The combinations include the state originating the speciation, and the states of the branches generated from the speciation. Fig. 1.3 shows an example of a two-states ClaSSE model for a tropical plant growth form.

Simulation function: `tree.classe()`

Parameters:

When adding more character states, the required parameters increase in accordance with the combinatorics of potential combinations. Specifically, let n denote the number of character states. Then, define the indices k, l, m as indicators for ancestral, transitional, and descendant character states respectively, assuming they take integer values between 1 and n inclusive. Now, introduce the following parameters:

- Rate of cladogenesis given the parent holds the lth character state, and daughters acquire the kth and mth character states (for distinct k, l, m).

$$\lambda_{klm} \quad \text{(transition from state } k \text{ to states } l \text{ and } m,\ 1 \leq k, l, m \leq n)$$

- Background extinction rate assigned to the kth character state.

$$\mu_k \quad (\text{for state } k,\ 1 \leq k \leq n)$$

- Transition rate from having the lth character state to acquiring the mth character state (applicable when $l \neq m$).

$$q_{lm} \quad (\text{from state } l \text{ to state } m,\ 1 \leq l \neq m \leq n)$$

Differential equations for the survival probabilities $P_k(t)$ and transitions between states:

$$\frac{dP_k(t)}{dt} = -(\mu_k + \sum_{l \neq k} q_{kl})P_k(t) + \sum_{l \neq k} q_{lk}P_l(t) + \sum_{l,m} \lambda_{klm}P_k(t)P_l(t)P_m(t)$$

By way of illustration, here are some concrete examples outlining the necessary parameters for varied character state counts:

One character state:

- λ_{111}: Rate of cladogenesis yielding offspring bearing the first character state.
- μ_1: Extinction rate applicable to the single character state.

Two character states:

- λ_{111}: Rate of cladogenesis generating progeny inheriting the first character state if the parent already possessed said attribute.
- λ_{112}: Rate of cladogenesis leading to descendants adopting the second character state when parents initially bore the initial feature.
- λ_{122}: Analogous scenario but with reversed roles in character acquisition order.
- λ_{211}: Another case wherein children adopt the first characteristic after the parent had shifted to the second configuration.
- λ_{212}: Identical context as above except the final outcome reflects the emergence of the second character state in offspring.
- λ_{222}: Parallel situation to the preceding cases yet with both parents and descendants uniformly exhibiting the latter trait.
- μ_1: Extinction rate attributed to the primary character state.

- μ_2: Loss rate ascribed to the secondary character state.
- q_{12}: Shift rate towards obtaining the second character state starting from the first.
- q_{21}: Reverse transformation rate moving away from the first characteristic toward attaining the second.

To reiterate, the cumulative quantity of parameters scales cubically with the count of character states (n^3); hence, careful deliberation should ensue whenever expanding beyond just a few states.

1.5. GeoSSE

The GeoSSE model (Geographic State Speciation and Extinction) combines features of the constant-rates birth-death model with a three-state Markov model. It differs from BiSSE in three specific ways [4]:

- The character states are defined so that a single, wide-ranging species may occupy more than one area or region simultaneously.
- The transitions between states are explicitly parameterized in terms of range expansion through dispersal and range contraction through local extirpation
- Rates of speciation, extinction, and dispersal are tied to the regions themselves rather than to the character states, causing the dynamics of wider-ranging species to be determined by the cumulative effects of the areas they inhabit.

GeoSSE operates with reconstructed phylogenies to estimate relationships among extant species. It considers speciation and global extinction as stochastic processes rather than assuming a fixed phylogenetic history.

Simulation function: `tree.geosse()`

Parameters:

- Speciation rates:

$$s_A \quad \text{(for area } A\text{)}$$

$$s_B \quad \text{(for area } B\text{)}$$

$$s_{AB} \quad \text{(for mixed distribution } AB\text{)}$$

- Extinction rates:

$$x_A \quad \text{(for area } A\text{)}$$

x_B (for area B)

- Dispersal rates:

d_A (from area A to area B)

d_B (from area B to area A)

Differential equations for the probabilities of survival and transition between geographic areas:

$$\frac{dP_A(t)}{dt} = -(s_A + x_A + d_A)P_A(t) + d_B P_B(t) + 2s_A P_A(t)P_A(t)$$

$$\frac{dP_B(t)}{dt} = -(s_B + x_B + d_B)P_B(t) + d_A P_A(t) + 2s_B P_B(t)P_B(t)$$

$$\frac{dP_{AB}(t)}{dt} = -s_{AB}P_{AB}(t) + s_A P_A(t)P_B(t) + s_B P_B(t)P_A(t)$$

1.6. QuaSSE

QuaSSE enables the specification and variation of speciation and extinction rates as arbitrary (user-defined) functions of trait values. This method significantly lags in speed compared to other methods in diversitree due to the necessity to solve a system of partial differential equations to compute the likelihood. This task is notably more complex than dealing with systems of ordinary differential equations or simple algebraic expressions in alternative methods. Although the basic interface resembles that of other methods, QuaSSE offers more options for controlling the integrator's behavior.

The tree simulation diverges slightly from other methods, primarily because there is no longer a canonical argument list, given that speciation and extinction rates are arbitrary functions of the character state.

Simulation function: `tree.quasse()`

Parameters: Functions defining speciation and extinction rates concerning quantitative traits:

- Speciation rate:

$\lambda(z)$

- Extinction rate:

$$\mu(z)$$

Differential equation for the probability of survival $P(z,t)$:

$$\frac{\partial P(z,t)}{\partial t} = -\lambda(z)P(z,t) + \frac{\partial}{\partial z}\left[D(z)\frac{\partial P(z,t)}{\partial z}\right] - \mu(z)P(z,t)$$

where $D(z)$ represents the dispersion or variance of the trait z.

1.7. BiSSEness

Binary-State Speciation and Extinction–node enhanced state shift (BiSSE-ness), estimates both the rate of change occurring along lineages and the probability of change occurring during speciation, as well as independent speciation and extinction rates for each character state [5].

For this reason, BiSSEness functions have p parameters, that deal with the probabilities of transitioning between different trait states in either a cladogenetic or anagenetic way. For example, p_{0c} represents the probability of a cladogenetic shift in the trait from state 0 to state 1 (trait change at speciation), while p_{0a} represents the probability of anagenetic shift state 0 to state 1 (trait change along lineage). The same goes for state 1.

Simulation function: `tree.bisseness()`

Parameters:

- Base speciation rates irrespective of character states.

$$\lambda_0 \quad \text{(for state 0)}$$

$$\lambda_1 \quad \text{(for state 1)}$$

- Baseline extinction rates regardless of character states.

$$\mu_0 \quad \text{(for state 0)}$$

$$\mu_1 \quad \text{(for state 1)}$$

- Transition rates between character states per unit of time.

$$q_{12} \quad \text{(from state 0 to state 1)}$$

$$q_{21} \quad \text{(from state 1 to state 0)}$$

- Probabilities of change along lineages:

$$p_{0c} \quad \text{(for state 0)}$$

$$p_{1c} \quad \text{(for state 1)}$$

- Probabilities of change during speciation:

$$p_{0a}$$

$$p_{1a}$$

Differential equations for the survival probabilities $P_i(t)$ for each state i:

$$\frac{dP_0(t)}{dt} = -(\lambda_0 + \mu_0 + q_{12})P_0(t) + q_{21}P_1(t) + 2\lambda_0 P_0(t)P_0(t) + p_{0a}(1 - P_0(t))$$

$$\frac{dP_1(t)}{dt} = -(\lambda_1 + \mu_1 + q_{21})P_1(t) + q_{12}P_0(t) + 2\lambda_1 P_1(t)P_1(t) + p_{1a}(1 - P_1(t))$$

1.8. Overview of models

Model	Parameters	Description
const BD	λ, μ	Models pure birth-death processes without character state transitions.
BiSSE	$\lambda_1, \lambda_2, \mu_1, \mu_2, q_{12}, q_{21}$	Constant rates of speciation, extinction, and transition between two character states.
MuSSE	$\lambda_1, \ldots, \lambda_K, \mu_1, \ldots, \mu_K, q_{ij}$	Generalizing BiSSE to K character states with constant rates.
QuaSSE	Arbitrary functions of trait values	Allows for variation of speciation and extinction rates as functions of trait values.
GeoSSE	$s_A, s_B, s_{AB}, x_A, x_B, d_A, d_B$	Accounts for geographical considerations with constant rates of speciation, extinction, and dispersal.
ClaSSE	λ_{klm} (where $1 \leq k, l, m \leq n$), μ_k (where $1 \leq k \leq n$), q_{lm} (where $1 \leq l \neq m \leq n$)	Distinguishes between anagenetic and cladogenetic speciation with variable rates. There will be n^3 cladogenesis rates and n extinction rates plus $n(n-1)$ transition rates, making a total of $n^3 + n + n(n-1)$ parameters.
BiSSEness	$\lambda_0, \lambda_1, \mu_0, \mu_1, q_{12}, q_{21}$, $p_{0c}, p_{0a}, p_{1c}, p_{1a}$	Estimates rates of change occurring along lineages and probabilities of change during speciation.

Examples of all these models, including the code used to generate trees and plots of the trees, can be found in the appendix, in the section "Examples" (.1).

2 | Simulations

Simulations are a crucial tool in evolutionary biology, allowing researchers to explore the effects of different parameters on the evolution of species. By simulating the evolution of species under different models, we can gain insights into the processes that have shaped the diversity of life on Earth. In this section, we will explore the use of simulations to study the evolution of species under different models of diversification and character evolution.

Gillespie's Stochastic Simulation Algorithm (SSA) is utilized across all models for simulating birth-death processes, with or without character state transitions, as well as incorporating geographical considerations and quantitative traits. Gillespie's algorithm enables the sampling of waiting times until the next event (speciation, extinction, or transition) from exponential distributions. It calculates the total rate of events and selects the actual event based on its relative proportion to the total rate. Additionally, each model outlines specific adaptations and extensions of Gillespie's algorithm to handle various aspects such as character state transitions, geographical factors, and arbitrary functions of quantitative traits.

Every step in chapter 3, 4 and 6 is reproducible: my repository with all the code can be found at `https://github.com/Mattia-Colbertaldo/PhyloML`.

2.1. Real data analysis for parameters range retrieval

The following analysis aims to choose the parameter ranges for the simulations. The 99 phylogenies analyzed included species of birds, crocodilians, turtles, amphibians, mammals, and squamates. The `make.bd` function - from `diversitree` package - was used, which is a Maximum Likelihood Estimation method to estimate the parameters of each real tree. The `make.bd` functions infers the parameters of the given tree following a constant rate birth-death model (CRBD), since it's not always possible to make inference

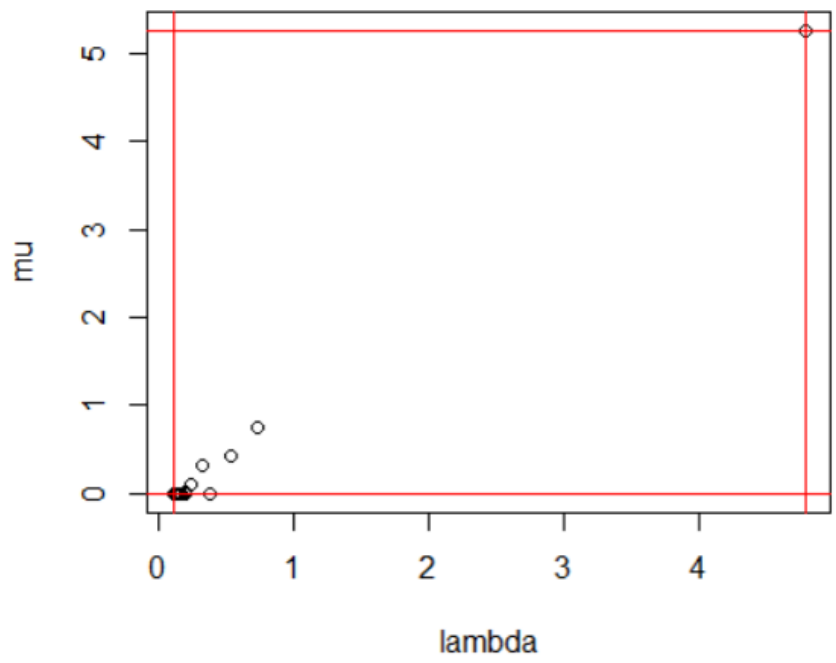


Figure 2.1: Speciation and extinction rates for real trees.

with other models because of the lack of analytical solutions. The resulting minimum and maximum parameters were:

- Speciation rate: 0.09924509 - 4.78357509
- Extinction rate: 0.000000 - 5.256081
- Number of species: 10 - 1000

Fig. 2.1 shows the distribution of speciation and extinction rates. After removing the most distant outlier, the definitive parameter ranges were:

- Speciation rate: 0.09924509 - 0.73201151
- Extinction rate: 0.000000 - 0.7530955
- Number of species: 10 - 1000

Many times, the generation of trees, based on the chosen parameter combination, produces trees with no species (null trees), because the species go extinct before they can generate other species. It is likely that this happens more frequently when the extinction

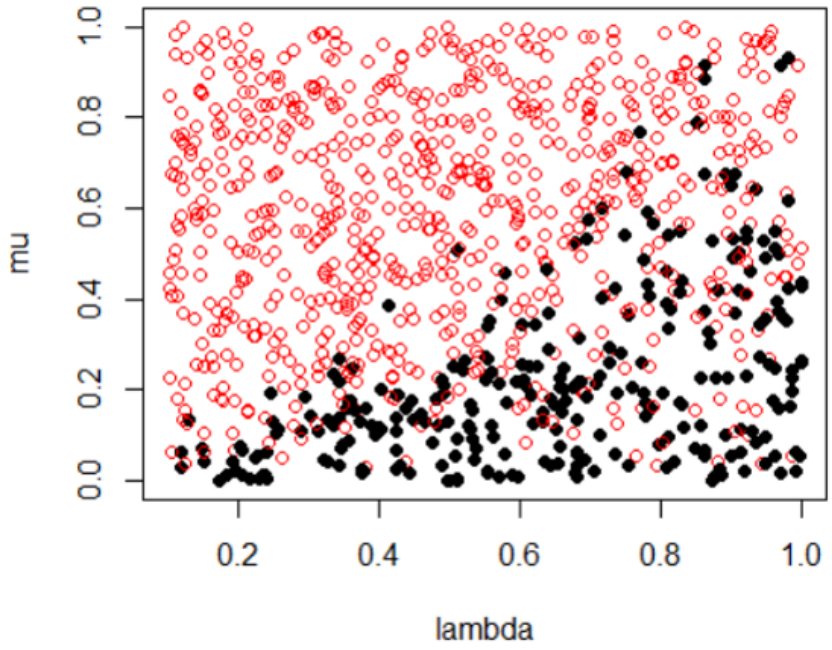


Figure 2.2: Trees generated with the constant birth-death model. Red points are null trees, while black points have at least one specie.

rate is higher than the speciation rate. With these results, the two parameter ranges are very similar, so it is likely that many generated trees are null. By generating 1000 trees with the constant BD model, setting `max.taxa = 30`, I obtained 725 / 1000 (72.5 %) null trees. Plotting which trees are null and which are not, it can be observed that every time the extinction rate is greater than the speciation rate, the generated tree is null (Fig. 2.2).

So it is reasonable to generate only trees with extinction rate less than or equal to speciation rate. In this case, only 293 / 1000 (29.3 %) trees are null.

2.2. Generating trees

I realized a script in R to generate trees for any of the previously mentioned models, customizing the parameters and the number of species. For each tree, the parameters are drawn from a uniform distribution within the intervals defined by the user. Similarly, to achieve a uniform distribution of the number of species in the trees, the maximum

number of taxa, `max.taxa`, is also drawn from a uniform distribution within the predefined interval. By plotting the number of species per tree, one can observe how the distribution can indeed be assumed uniform, considering the limited number of generated trees (Fig. 2.3).

The code saves the trees to a file, in Newick format, and also saves the parameters to another file.

Even though I implemented all the code to correctly simulate QuaSSE trees, it was decided to ignore it in the next sections, for two main reasons: firstly, it is extremely slow to simulate, and i had limited computational resources, but most importantly, the space of choice of the functions that produce the rates is infinite, so it's not suitable for an inference application. A few functions were included inside the previously provided repository, but, after a further work of selection of the main functions governating speciation and extinction rates in QuaSSE models, it would be possible to use this model as well as the other ones.

2.3. Other packages

In the process of selecting the most suitable method for simulating phylogenetic trees, various packages and functions were explored. This exploration involved testing different functions from packages such as `ape`, `geiger`, `TreeSim`, `TreeSimGM`, `phytools`, and `diversitree`. Computational time and the distribution of the number of species generated at the end of the tree were analyzed. It was found that the `diversitree` package provided the most comprehensive and customizable set of functions for simulating trees under different models. Therefore, it was selected for further analysis and implementation in this study. In the previous section, we analyzed all the phylogenetic models offered by the diversitree package. In the following one, we will explore the other packages and functions that were tested during the selection process.

2.3.1. `ape` package:

These three functions simulate phylogenies under any time-dependent birth–death model.

- `rlineage`: Simulates a complete tree including extinct species (Fig. 2.4).
- `rbdtree`: Generates an ultrametric tree with only present species.
- `rphylo`: Generates a tree with a fixed number of present species.

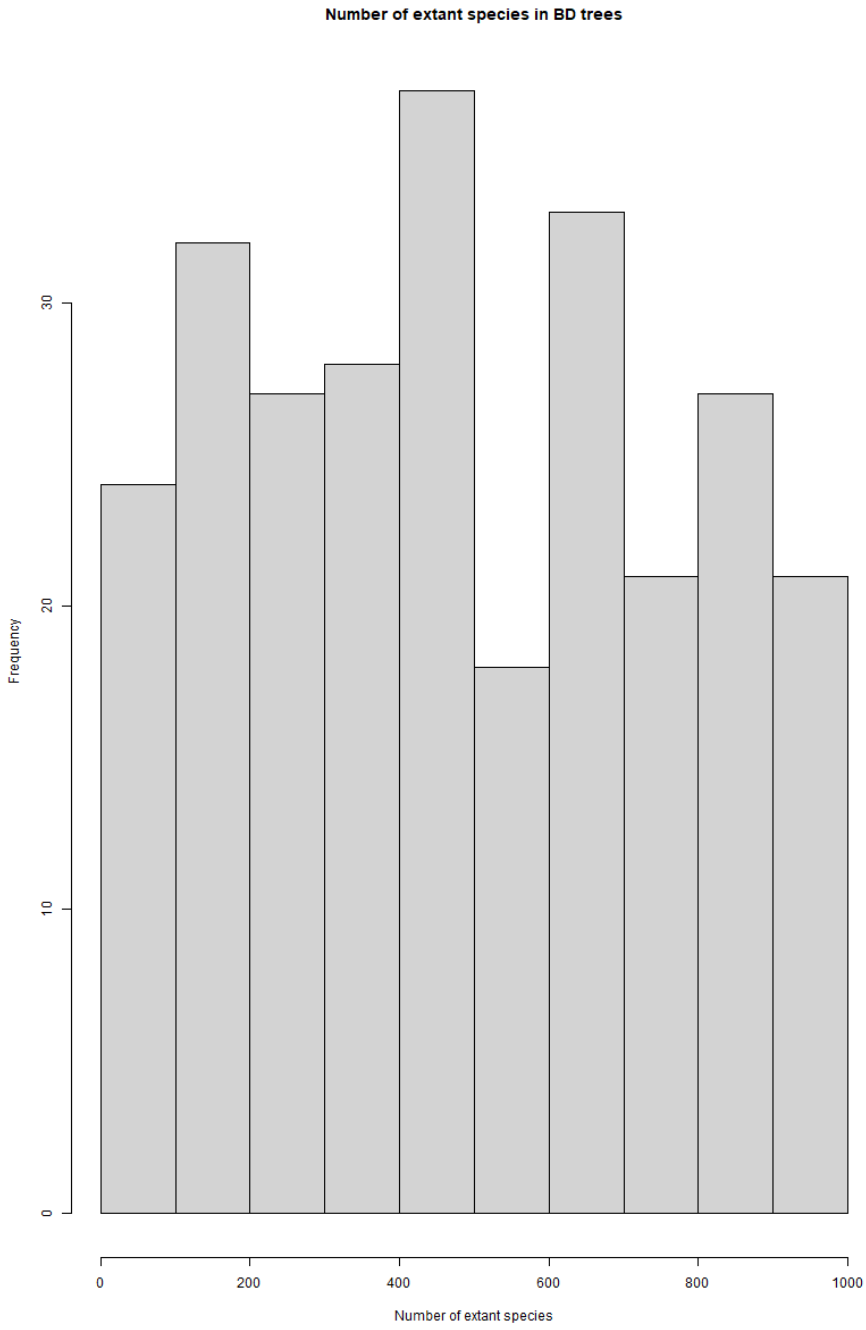


Figure 2.3: Number of species in each tree, generated with the function tree.bd. The number of species is uniformly distributed between 10 and 1000.

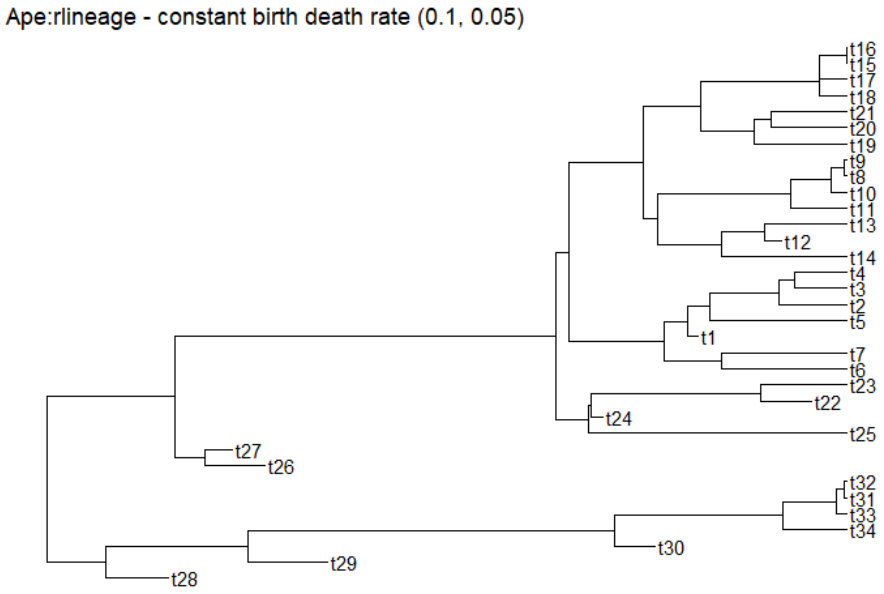


Figure 2.4: Simulation of a phylogenetic tree with the function rlineage, following, in this case, a constant birth-death model. Extinct species are reported. Speciation rate is set to 0.1, while extinction rate is 0.05.

2.3.2. `geiger` package:

A package for macroevolutionary simulation and estimating parameters related to diversification from comparative phylogenetic data.

- `sim.bdtree`: Simulates the growth of a phylogenetic tree under a uniform, time-homogeneous birth-death process. This means that every lineage has a constant probability of speciating, and a constant probability of going extinct, per unit time.
- If `extinct=FALSE`, the function builds trees until one has at least one surviving lineage.

2.3.3. `TreeSim` package:

- `sim.bd.taxa`: Simulates trees on n species under the constant rate birth-death process with options for incomplete sampling.
- `sim.rateshift.taxa`: Simulating trees incorporating mass extinction events and rate shifts.

  In Figure 2.5, an example of rate shifting can be observed, attributed to catastrophic events and subsequent rate shifts. The implemented properties are as follows:

- Speciation rates: Randomly drawn from a uniform distribution between 1 and 2 for each time step.
- Extinction rates: Sampled for each time step from a uniform distribution between 0 and lambda at the corresponding time step.
- Proportion of species surviving mass extinction events: Randomly drawn from a uniform distribution between 0.1 and 0.3 for each event.
- Times of mass extinctions and rate shifts: A given number of events (20 in this case) are randomly distributed along time.

2.3.4. `TreeSimGM` package:

Simulating General Model Trees:

- `sim.taxa`: Simulates trees on n extant species under a general model, allowing stopping after a given number of co-existing taxa or sampling from a larger tree.
- `sim.age`: Generates stochastic trees based on extant tips' stem age under a general model, accommodating various speciation and extinction distributions.

2.3.5. `phytools` package:

- `pbtree`: Simulates stochastic birth-death trees, with options for conditioning on taxa number, time, or both simultaneously. Allows continuous or discrete time simulations, with discrete-time allowing multiple speciation events on separate branches. An example is depicted in figure 2.6.

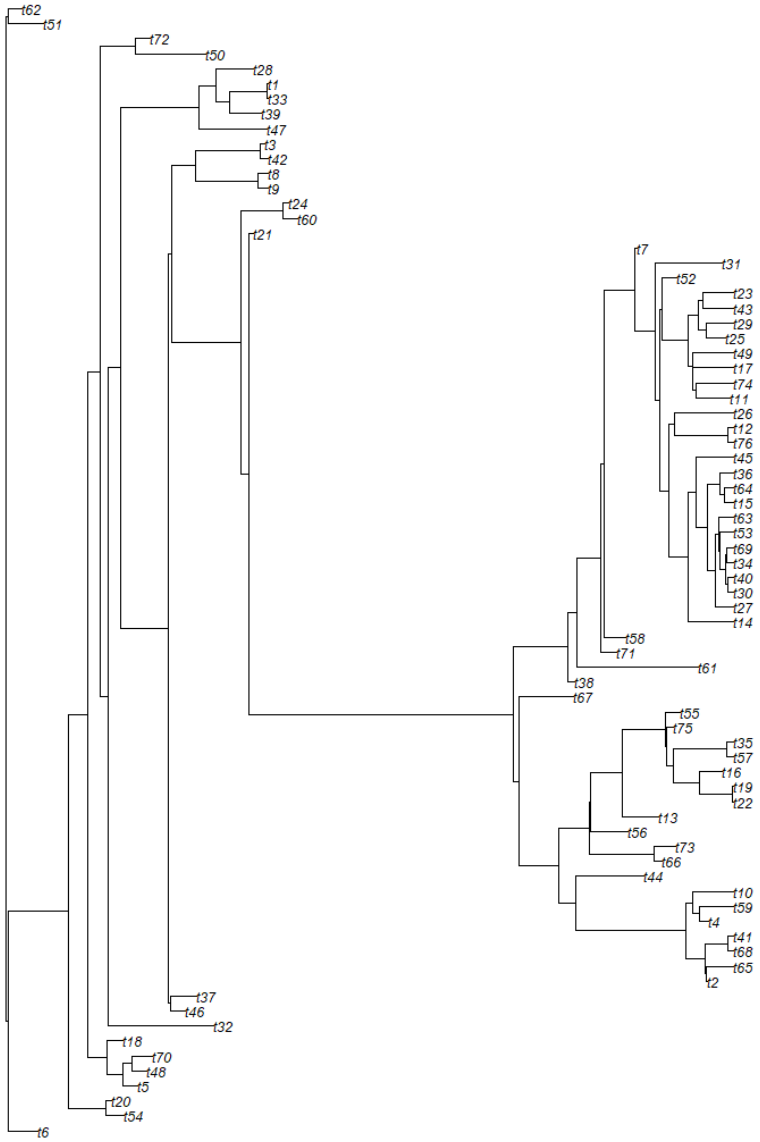


Figure 2.5: This tree was generated with the function sim.rateshift.taxa, from the package TreeSim. We can see how irregular the pattern is, compared to the constant birth-death models seen before, due to the presence of mass extinction events and rate shifts.

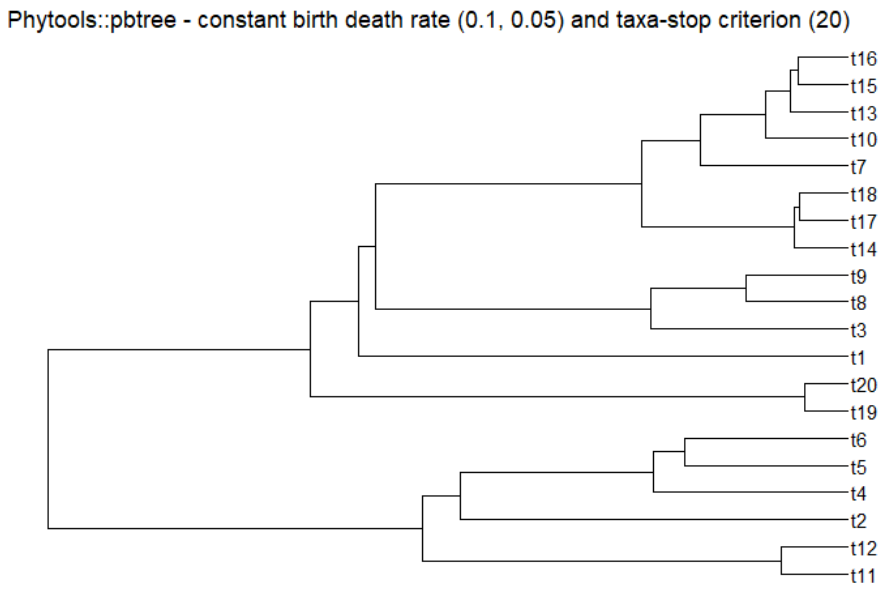


Figure 2.6: Simulated phylogenetic tree using phytools::pbtree, based on a desired number of extant species (i.e., taxa-stop criterion). Extinct species are omitted. Speciation and extinction rate are constant, respectively 0.1 and 0.05, while the desired number of extant species at present time is set to 20.

3 | Extending Deep Learning Techniques to Complex Phylogenetic Models

This paper[7] introduces a novel approach to diversification inference, based on deep learning. The method is applied to the time constant homogeneous birth-death model and the Binary-State Speciation and Extinction model. The authors show the results obtained with the deep learning models trained on the CDV encoding and the SS encoding, and with the MLE method: an error boxplot is shown in Fig. 3.1 for BD model.

The models are trained to learn the relation between simulations and parameter values as a regression problem. The approach is demonstrated to be accurate and time-efficient, providing a promising new inference approach for future models in the field. The authors provided a repository with the code used in the paper (available at the following link: `https://github.com/JakubVoz/deeptimelearning`); it includes:

- `Simuators`: I kept using my own code to generate trees for my target models.
- `Encoders`: These python files can be used to encode the trees in two formats:
  - `Complete Diversity-reordered Vector (CDV)`: it is an adaptation of g the CBLV tree representation from Voznica et al. (2022) to ultrametric phylogenies and to ultrametric phylogenies with tip state data.
  - `Summary Statistics (SS)`: it is a set of summary statistics that can be used to encode the phylogenies.

  The necessary files to reproduce these steps can be found at `https://github.com/Mattia-Colbertaldo/PhyloML`.

  These codes were modified to adapt them to our trees, since the authors included only trees with a length inferior to 500, but, analyzing real trees, I found out that

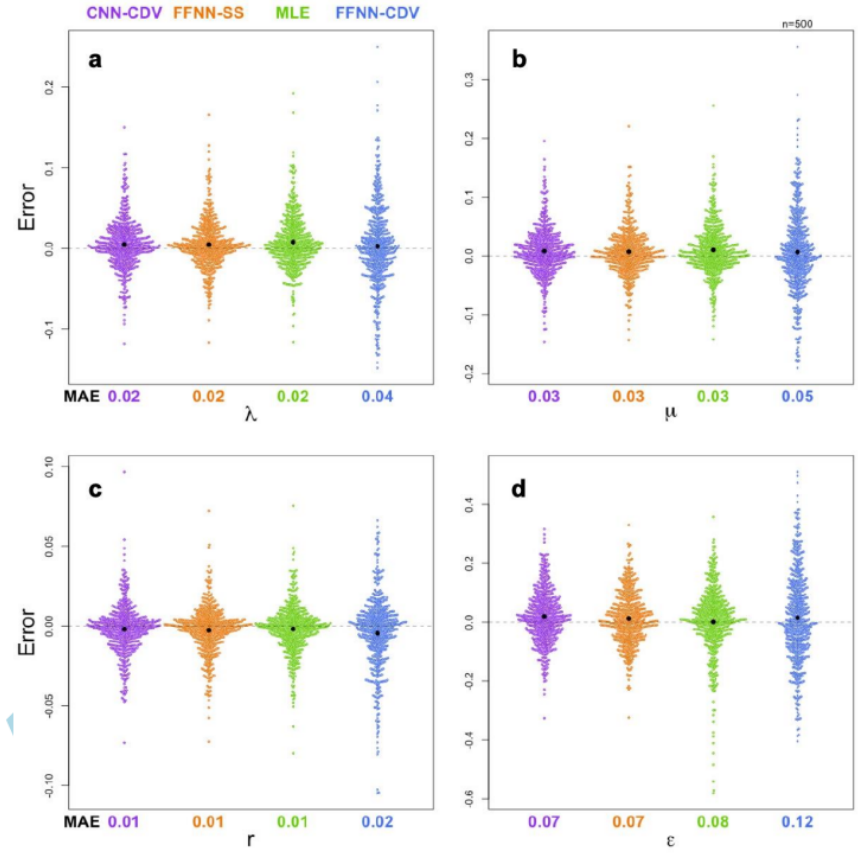


Figure 3.1: Results obtained on BD with the deep learning models trained on the CDV encoding and the SS encoding, and with the MLE method. The plots show the Mean Absolute Error (MAE) for the speciation rate, the extinction rate, net diversification and turnover rate.

the length of the trees can be much higher, so the maximum length was set to 1000. Another change was to write the output to a csv file row by row, instead of writing the whole matrix at once, that could result in memory issues. The main improvement was addressing the ill-posedness of the problem with a transformation of the data, which will be discussed in the next section.

- `Models`: This folder contains the deep learning models used in the paper. But, since our trees are much longer than the ones used in the paper - as I wished for real world applicability - it was not possible to compare the obtained results with the ones in the paper.
- `Notebooks`: This folder contains the jupyter notebooks modified (to be suitaused to train the models. These notebooks were modified and used to train our models.

The goal was to train various deep learning models on the trees generated with the models I previously discussed to accurately infer the parameters of the models.

Additionally, I introduced a Random Forest Classifier to the models, that led to the best results, as shown in the following sections.

3.1. Addressing Ill-Posedness of the Problem

In this section, we address the challenge of ill-posedness inherent in the parameter estimation problem. Specifically, we encounter a scenario where multiple combinations of the speciation rates, denoted as lambda (λ), extinction rates, denoted as mu (μ), and transition rates, denoted as (q), can produce the same tree topology. This ambiguity impedes the model's ability to effectively learn the relationship between the trees and the parameters.

To mitigate this issue, we have implemented a reordering strategy for the parameters associated with each tree. The reordering is based on the lambda values, ensuring that the parameters are consistently arranged in increasing order of lambda. By imposing this ordering constraint, we provide the model with a more structured input, facilitating its learning process.

Through this approach, we aim to enhance the model's capacity to discern meaningful patterns and relationships between the tree topologies and the corresponding parameters. By organizing the parameters in a systematic manner, we strive to improve the robustness and accuracy of the parameter estimation process in the face of the inherent ill-posedness of the problem.

3.2. Training Results

In extending the methodology from the initial paper to the frameworks of MuSSE, GeoSSE, BiSSE, and ClaSSE models, the results were notably positive in terms of parameter estimation. The applied techniques exhibited a high degree of accuracy in estimating most parameters across these complex models, demonstrating the versatility and robustness of the original methods when adapted to these new contexts.

The effectiveness of these methods in providing reliable parameter estimates across diverse evolutionary models underscores their potential for broad application. This success is indicative of the strong foundational approach established in the initial study, which has now been validated and expanded upon in this extended analysis.

Overall, the results reinforce the utility and adaptability of the original methodology, showcasing its capability to handle a variety of model structures with precision and reliability. This accomplishment marks a significant step forward in parameter estimation within the fields encompassed by MuSSE, GeoSSE, BiSSE, and ClaSSE, setting a solid groundwork for future research and applications.

The encoding method did not significantly impact the results, as both the CDV and SS encodings yielded comparable performance across the models. This consistency suggests that the deep learning models are robust to variations in encoding strategies, providing a reliable framework for parameter estimation in diverse evolutionary contexts.

As said before, the Random Forest Classifier was introduced in the field: a Random Forest Classifier works by constructing multiple decision trees during the training phase and outputting the mode of the classes (classification) or the mean prediction (regression) of the individual trees. It is possible to control the number of trees used in the forest, as well as the maximum depth of the trees. It demonstrated promising results, as shown in Fig. 3.2, offering an alternative approach to parameter estimation that complements the deep learning models. The classifier's ability to accurately predict parameters across the different models highlights its versatility and effectiveness in handling complex evolutionary scenarios. The error is notably low, even considering the low number of trees used for the training, and the training of this model (100 estimators and a maximum depth of 10) was much faster than the training of the deep learning models. Its performance with BiSSE model in predicting speciation and extinction rates is similarly impressive (Fig. 3.3), considering the limitation on the number available trees. In addition, the distribution of the

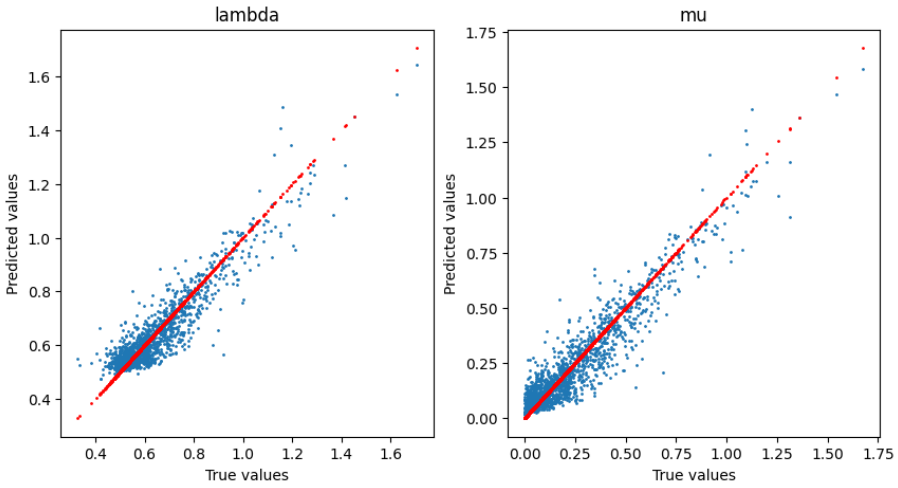


Figure 3.2: Results obtained with the Random Forest Classifier for the BD model. The red scatter points represent the true values, while the blue points represent the predicted values. The points are really close to the line $y = x$, which means that the predictions are accurate.

parameters is not uniform (as shown in Fig. 3.4 for the MuSSE model), the predictions are biased towards the mean of the parameters. A possible solution to this issue could be to cut out most of the trees with parameters close to the mean, but the available data is already limited, due to the high computational cost of the simulations, so this solution is not feasible.

The difficulty in predicting the transition rates is likely due to the fact that the transition rates are not directly related to the tree topology, but to the character states of the species, and tree structures do not contain information about the character states, so many more data would be needed.

Non-Recurrent Neural Networks performed similarly among themselves, and notably the single Dense Layer model showed a slightly better performance in every case.

The Recurrent Neural Networks models (GRU and LSTM) couldn't learn any relation between the trees and the parameters, as shown in Fig. 3.5 for the BiSSEness model. The reason is that Summary Statistics encoding is not sequential.

To train the models, the data was firstly splitted in training and test set, keeping 20% of the data for the test set; then 20% of the training set was kept for the validation.

An additional technique consisted in training the models with the individual states of the parameters, to see if the results could improve. For each of the non-recurrent neural network models, four additional models were trained, one for each individual state: one

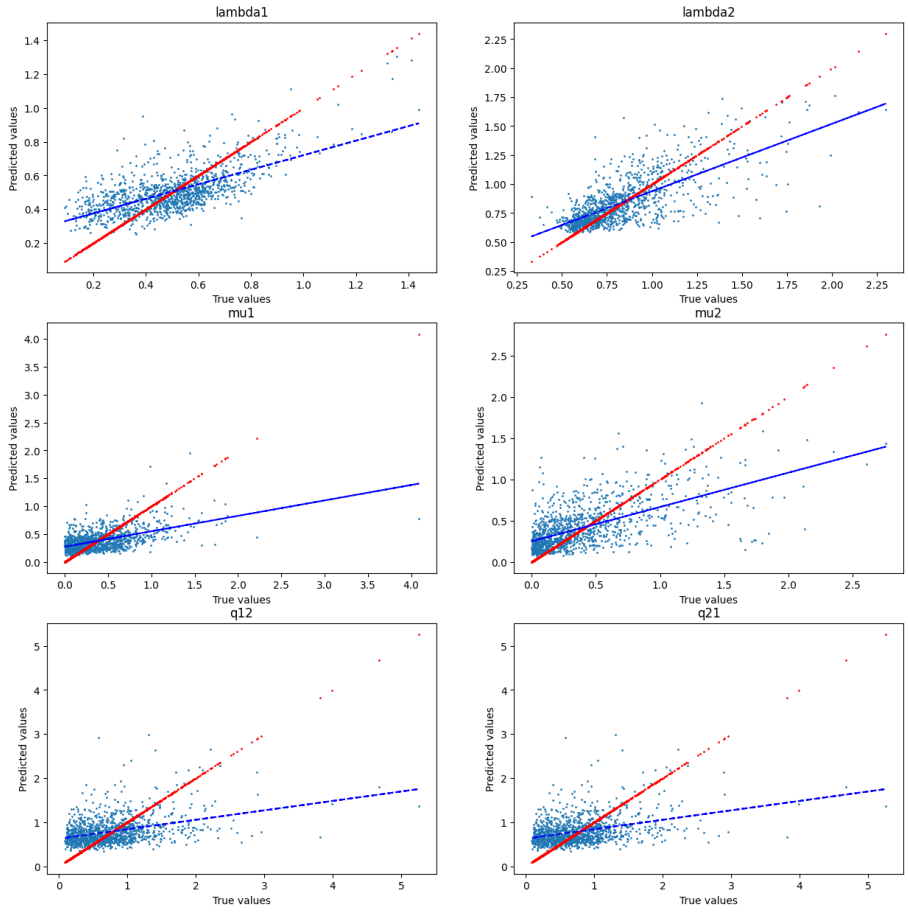


Figure 3.3: Predictions obtained with the Random Forest Classifier for the BiSSE model. Predictions are accurate, considering the limitation on the number available trees and the non-uniformly distribution of training data, that lead to notable a bias towards the mean.

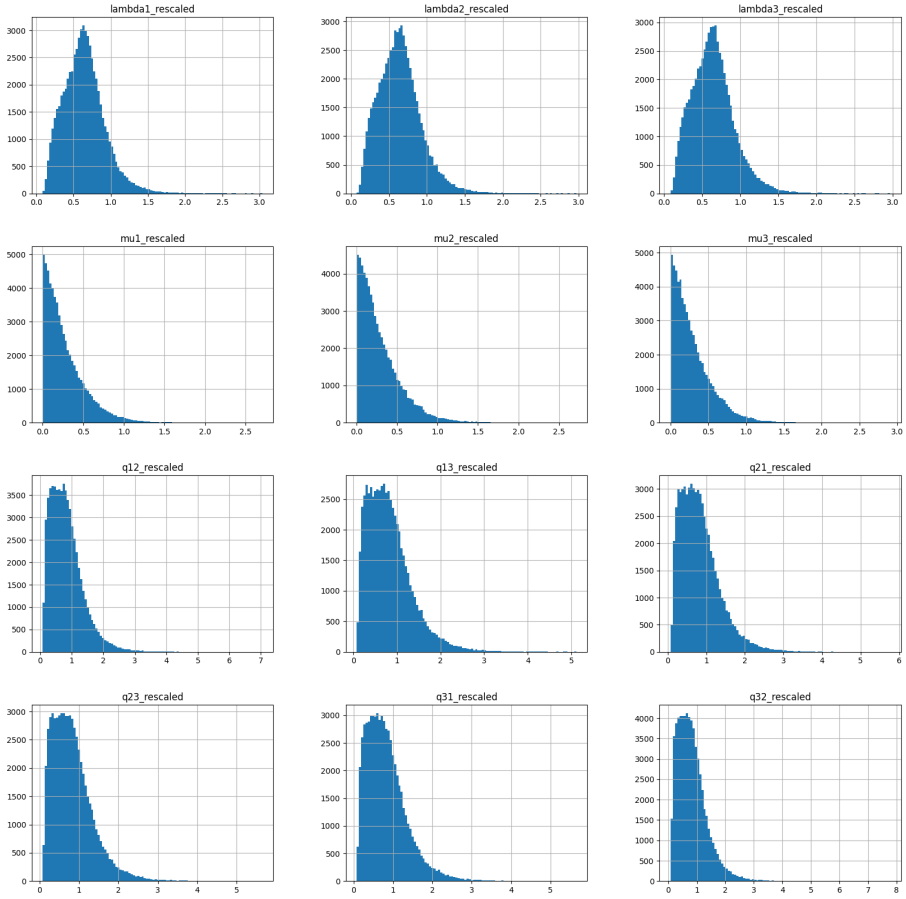


Figure 3.4: Target parameters for the MuSSE model. Note that these values have already been rescaled. As one may expect, due to the non uniform distribution of the parameters, the predictions will be biased towards the mean. Unfortunately, due to the limitation in computational resources, I couldn't cut out any value to make the distribution uniform, since the trees are already too few.

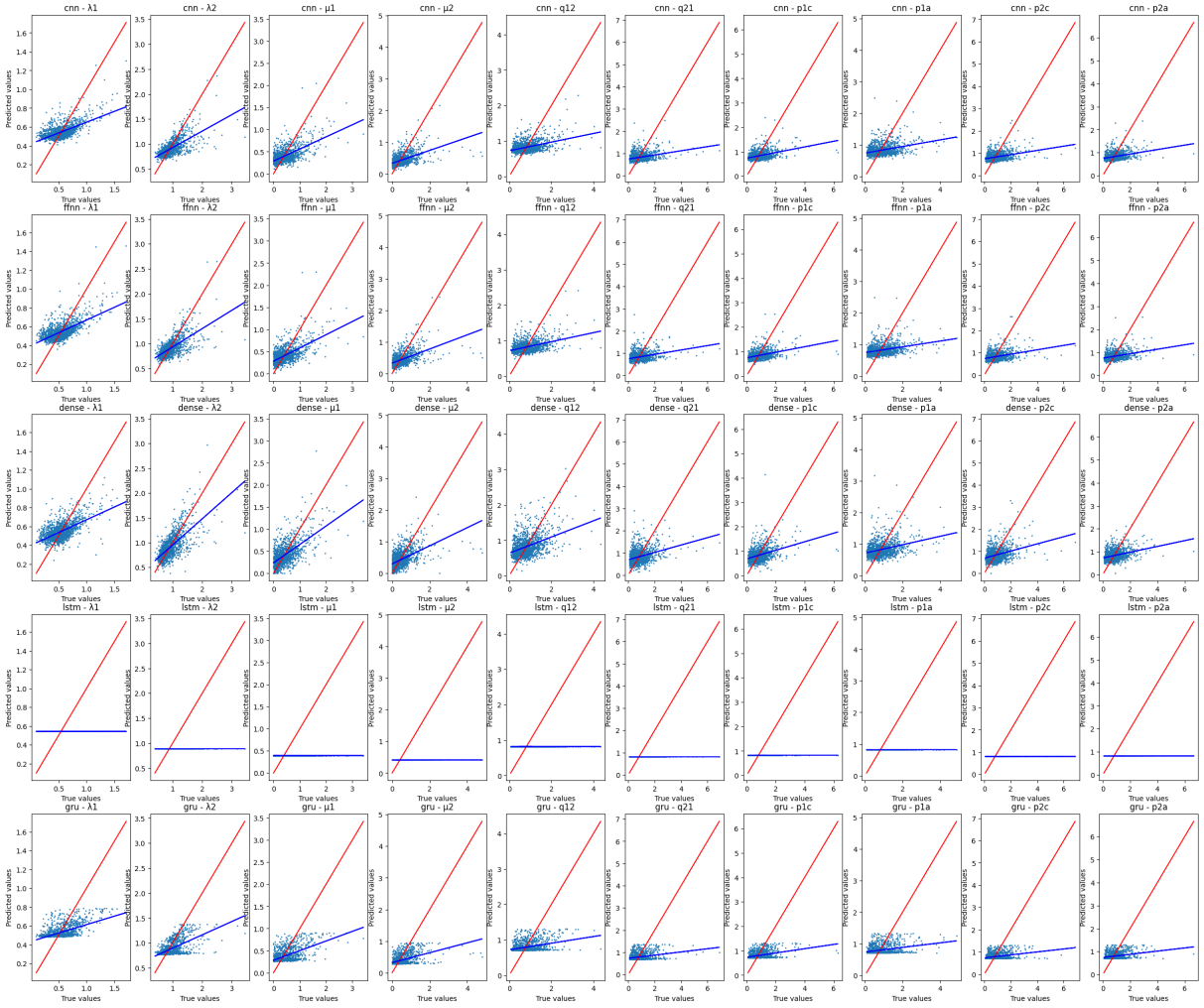


Figure 3.5: BiSSEness model, one of the most complex to infer. The state-related parameters (from the 5th column on) are difficult to be predicted due to the already discussed reasons. Notably, RNN models (GRU and LSTM, the last two rows) have the worst performance, because Summary Statistics encoding is not sequential.

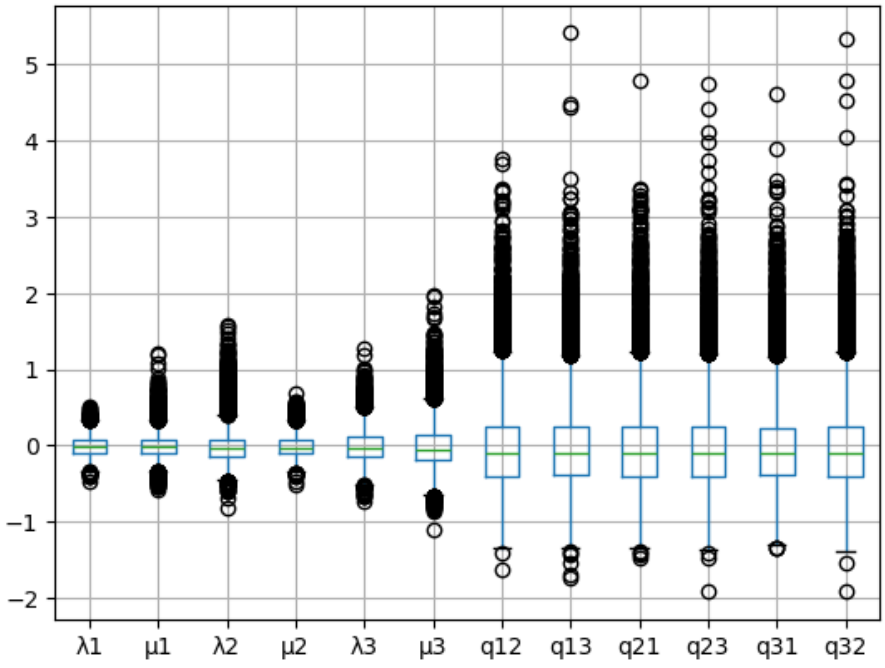


Figure 3.6: Boxplot of the error of the predictions obtained with the Random Forest Classifier for the MuSSE model. As said before, the classifier lacks in predicting the transition rates, due to tree structures not containing information about the character states and the limitation of available data , while the speciation and extinction rates are predicted with a good accuracy.

model to predict λ_1 and μ_1, one for λ_2 and μ_2, another one for λ_3 and μ_3, and then a model for the transition rates q_{12}, q_{13}, q_{23}, q_{21}, q_{31}, q_{32}. However, the results were almost identical to the ones obtained with the standard models predicting all the parameters at once.

The performance of the Random Forest Classifier was also evaluated for the other multi-state models, and the results stayed consistent: the difference in accuracy with deep learning models was basically null, being able to perform well in predicting the parameters of the models, exception made for the transition rates. In Fig. 3.6, I reported the boxplot of the error obtained with the Random Forest Classifier for the MuSSE model.

With GeoSSE model, both the deep learning models and the Random Forest Classifier showed a good performance, as shown in Fig. 3.7. The Random Forest Classifier performed, ad usual, very similarly to the single Dense Layer model, that was also the best performing model among the non-recurrent neural networks. In terms of parameters, GeoSSE is basically the BiSSE model plus a speciation rate among the two geographical regions (that we can treat as a character state). So the results obtained with the GeoSSE

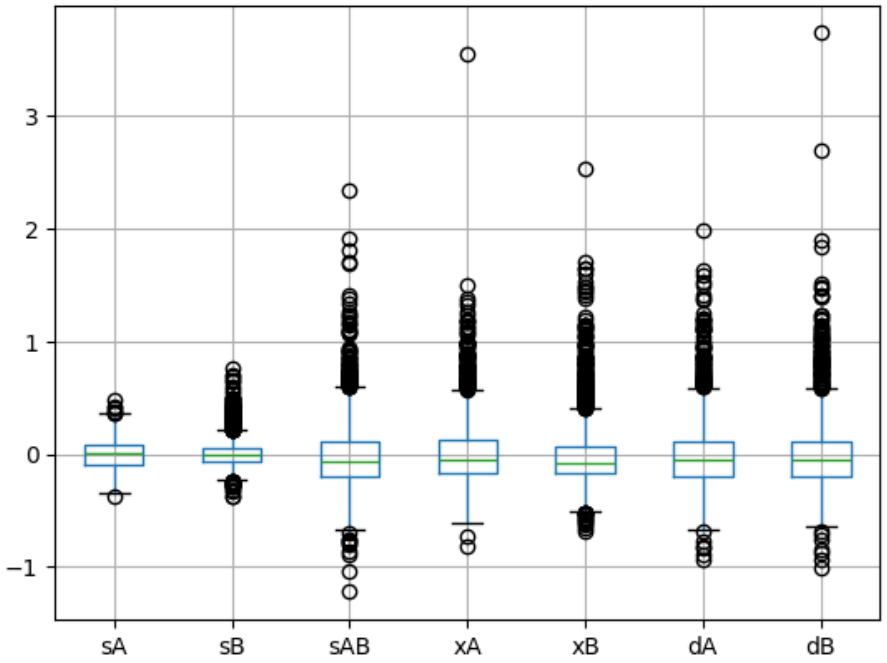


Figure 3.7: Boxplot of the error of the predictions obtained with the Random Forest Classifier for the GeoSSE model. The classifier performed well in predicting the parameters of the model. GeoSSE is basically the BiSSE model plus a speciation rate among the two geographical regions, so the results were very similar to the ones obtained with the BiSSE model.

model were very similar to the ones obtained with the BiSSE model.

BiSSEness model was, instead, more challenging to predict. This is a model that estimates rates of change occurring along lineages and the probability of change occurring during speciation, as well as independent speciation and extinction rates for each character state. So it is BiSSE (6 parameters) plus 4 parameters for the probabilities of shifting character states during speciation or lineage development, for a total of 10 parameters. As said before, tree structures do not contain information about the character states, so the transition rates are not predicted as well as the other parameters, and the same goes for the probabilities of shifting character states during speciation or lineage development.

ClaSSE model was the most difficult to predict, as shown in Fig. 3.8. ClaSSE is a model that distinguishes between anagenetic and cladogenetic speciation with variable rates. The ClaSSE model has $3n$ cladogenesis rates, n extinction rates, and $n(n-1)$ transition rates, making a total of $3n + n + n(n-1)$ parameters. For instance, for a model with 2 character states, there will be 6 cladogenesis rates, 2 extinction rates, and 2 transition rates, for a total of 10 parameters. But since tree structures do not contain

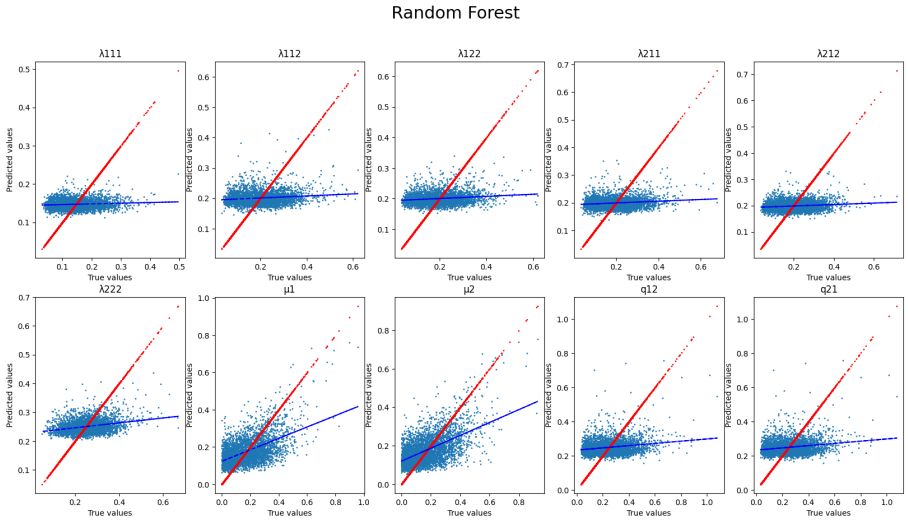


Figure 3.8: Results obtained with the Random Forest Classifier for the ClaSSE model. The red scatter points represent the true values, while the blue points represent the predicted values. Cladogenetic speciation rates and transition rates can't be predicted, due to the fact that tree structures do not contain information about the character states. Extinction rates, instead, are predicted with a good accuracy, as they do not depend on the character states (row 2, columns 2 and 3).

any information about the character states, it's difficult for the model to distinguish if a speciation event is cladogenetic or anagenetic, and if a cladogenesis (speciation) led to state 0 or 1; so the transition rates are not predicted as well as the other parameters. Extinction rates, instead, do not depend on the character states, so they are predicted with the same good accuracy as the other models, as shown in Fig. 3.8.

3.3. Conclusions

The results obtained with the deep learning models (FFNN, CNN, and Linear) were promising, demonstrating these models' ability to accurately predict the parameters of the phylogenetic models. The Random Forest Classifier, in particular, exhibited exceptionally accurate results and proved to be the most effective approach for parameter estimation, with the added advantage of faster training times.

Predicting the individual states of parameters for the MuSSE model yielded results that were similar or worse compared to predicting all parameters simultaneously. This suggests that the parameters are not independent, and failing to consider their interrelationships can lead to suboptimal predictions. Recurrent Neural Networks models (GRU and LSTM), on the other hand, were unable to learn any meaningful relationships between

the trees and the parameters.

A single Dense Layer model consistently showed slightly better performance in every scenario, closely rivaling the Random Forest Classifier. Given its faster training time compared to other models, including the Random Forest Classifier, the Dense Layer model emerges as a viable choice for this type of problem. The encoding method (CDV vs. SS) did not significantly impact the results, with both encodings yielding comparable performance across the models.

The distribution of parameters generated by the simulations was not uniform, leading to biased predictions towards the mean of the parameters. Addressing this issue could be an area for further work, especially if more data become available. The models were trained on a relatively small dataset due to limited computational resources, and the results could likely be improved with training on a larger dataset.

A notable limitation is that tree structures do not contain information about the character states of the species, so many more trees are needed to infer state-related parameters. Consequently, the models cannot accurately predict transition rates or the probabilities of character state shifts during speciation or lineage development. This issue is particularly relevant for models dependent on character states, such as BiSSEness and ClaSSE. The results obtained with these models, and for transition rates in general, reflect this limitation. Future work could explore alternative encoding methods that incorporate character states, enhancing the accuracy of predictions for transition rates and probabilities of character state shifts during speciation or lineage development.

In conclusion, the Random Forest Classifier was the best-performing model, demonstrating good accuracy in predicting parameters compared to MLE results provided by the authors, with remarkably short training times.

4 | Impact of parameter constraints and data manipulation on the results

The paper "A Comparison of Deep Learning Architectures for Inferring Parameters of Diversification Models from Extant Phylogenies" [9] explores the use of deep learning for phylogenetic inference, with a focus on understanding the ideal network architecture and data representation for using deep learning in phylogenetic inference. The authors evaluate the performance of different neural network architectures (DNN, CNN, RNN, GNN) and phylogeny representations (summary statistics, Lineage Through Time or LTT, phylogeny encoding, and phylogeny graph) for inferring rates of the Constant Rate Birth-Death (CRBD) and the Binary State Speciation and Extinction (BISSE) models. The authors find that deep learning methods can reach similar or even higher accuracy than Maximum Likelihood Estimation, provided that network architectures and phylogeny representations are appropriately tuned to the respective model. For example, for the CRBD model, the authors find that CNNs and RNNs fed with LTTs outperform other combinations of network architecture and phylogeny representation, presumably because the LTT is a sufficient and therefore less redundant statistic for homogenous BD models. For the more complex BiSSE model, however, it was necessary to feed the network with both topology and tip states information to reach acceptable performance. Overall, the results suggest that deep learning provides a promising alternative for phylogenetic inference, but that data representation and architecture have strong effects on the inferential performance.

A comprehensive image, showing total errors for the different models and phylogeny representations, is shown in Fig. 4.1.

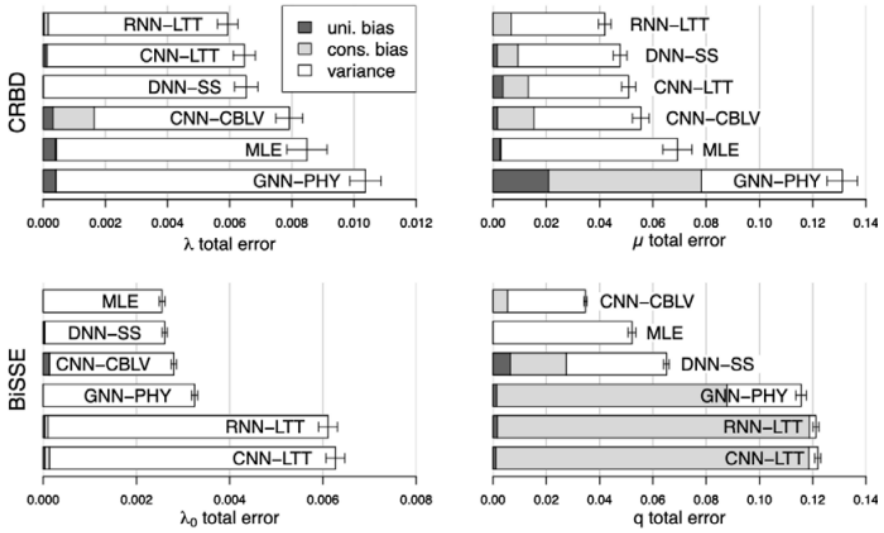


Figure 4.1: Paper results obtained with the different models and phylogeny representations.

4.1. Available configurations

I proceeded to train the models with the different architectures and phylogeny representations used in the paper, to see if the results I obtained were similar to the ones obtained by the authors. I first got the code from the repository provided by the authors, but there were some non available sections:

- CNN with LTT encoding (empty)
- RNN with LTT encoding (empty)
- CNN with CBLV encoding (not working)
- GNN (empty)
- MLE (empty)

I tried to make the CNN with CBLV encoding work, but I couldn't, so I decided to train the models with the only available configuration, that is DNN with SS encoding.

The repository provides code to generate the trees, and to convert the trees to the different encodings used in the paper.

4.2. Parameter Constraints

The authors imposed four constraints on the model to simplify inference, thus decreasing the number of free parameters from six to two. Specifically, the constraints for the BiSSE model are:

1. $\lambda_1 = 2\lambda_0$
2. $\mu_0 = 0$
3. $\mu_1 = 0$
4. $q_{01} = q_{10}$

Constraint 1 ensures that states 0 and 1 have different speciation rates, 2 and 3 make the model pure birth, and 4 is an assumption of symmetry that makes the probabilities to switch from one state to another equal. The authors simplified the model to focus on the comparison of network architectures and data representation.

This simplification will naturally increase the accuracy of the predictions, since the number of parameters to predict is drastically reduced, and the constraints make the model more predictable.

4.3. Results

To train the neural networks, the authors simulated 100,000 phylogenies for the CRBD model and 1,000,000 phylogenies for the BiSSE model using the Diversitree library [8]. Without having access to the same computational resources, I trained the models with 1000 trees for the CRBD model and 1000 trees for the BiSSE model. The lower number of trees with respect to my previous experiment is due to the fact that the code provided by the authors to generate trees is much more computationally expensive.

Following the constraints imposed by the authors on the BiSSE model, I imposed the same constraints on the MuSSE model:

1. $\lambda_1 = 2\lambda_0$
2. $\lambda_2 = 3\lambda_0$
3. $\mu_0 = 0$
4. $\mu_1 = 0$
5. $\mu_2 = 0$

6. $q_{01} = q_{10}$
7. $q_{02} = q_{20}$
8. $q_{12} = q_{21}$

The results of the MuSSE model with only 1000 trees, DNN architecture, and SS encoding are shown in Fig. 4.2.

Seeing the incredibly good results obtained with the MuSSE model, with a similar architecture to the ones used in the previous experiment, but especially with only 1000 trees, a deeper analysis of the code provided by the authors was necessary.

4.4. Artificial inflation of the results

Upon further examination of the code provided by the authors, It has come to our attention that the true values were incorporated into the training set, thereby inflating the results artificially.

In particular, the file `use-cases/02_convert-phylogeny.Rmd`, as the name suggests, converts the phylogeny to the different encodings used in the paper, while files named `use-cases/03*-predict-*.Rmd` load the Rds files generated by the previous script and create the training and testing sets.

For Summary Statistics and Lineage Through Time, the authors used the following code to generate the encoding:

```
# Summary Statistics:
sumstat <- generateSumStatFromPhylo(phylo, true_params)
saveRDS(sumstat, paste("data/phylogeny-", model, "-",
expname, "-sumstat.rds", sep=""))
```

And if we go inside the function `generateSumStatFromPhylo`, located in the file `R/convert-phylo-to-sumstat.R`, we can see that those true values, passed as arguments, are added to the dataset:

```
generateSumStatFromPhylo <- function(trees, true.param){
```

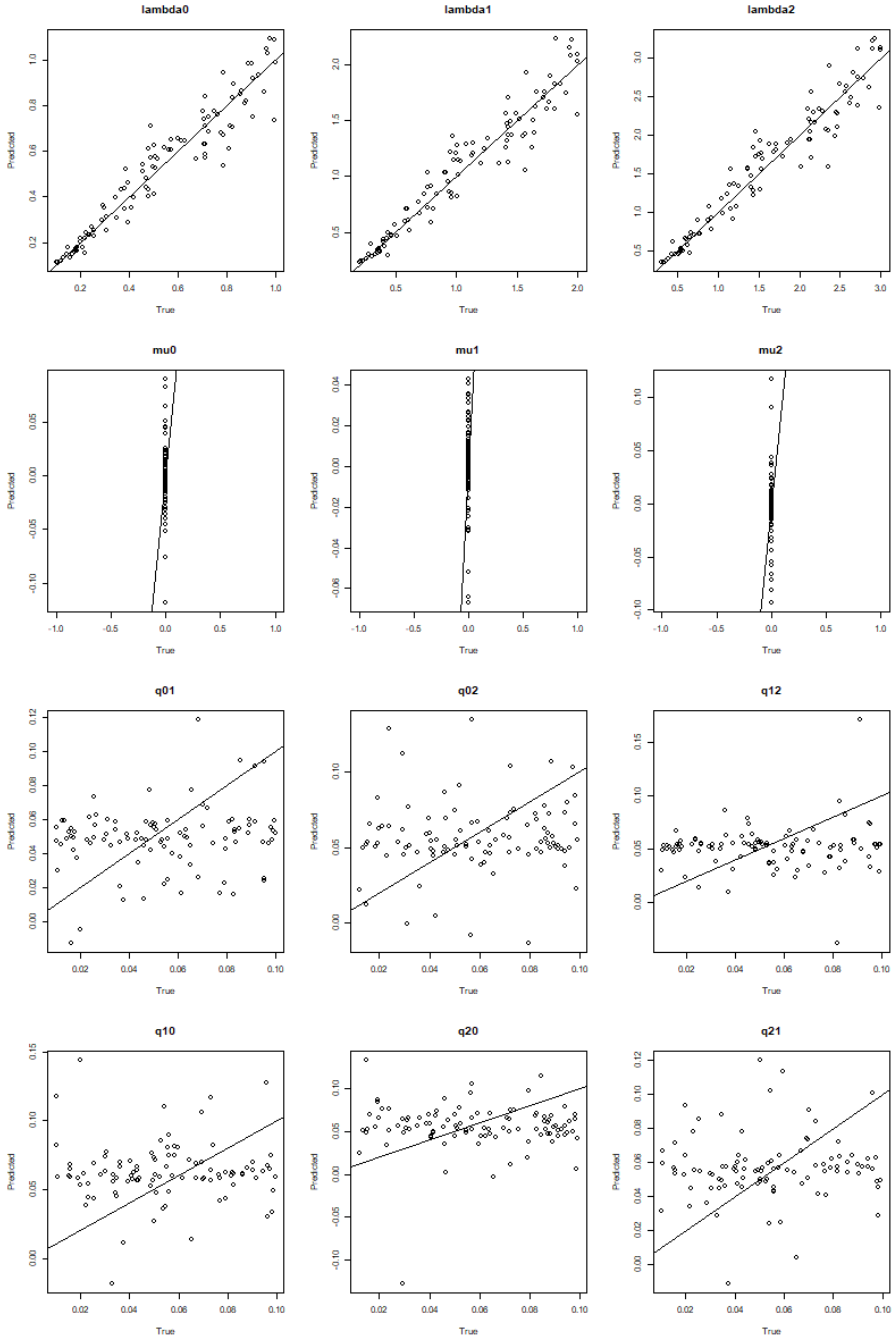


Figure 4.2: MuSSE with 3 states: True vs predicted parameters with DNN architecture and SS encoding. Only 1000 trees were used to train the models. The black line represents the true values.

```
    ss.names <- create_ss.names() # summary statistic names
    df <- create_ss_dataframe(ss.names) # initialize the data.frame (empty)

    n_trees <- length(trees)

    cat("Computing tree summary statistics...\n")

    for (i in 1:n_trees){
    tree <- trees[[i]] # get the tree
    ss   <- get_ss(tree) # compute summary statistics
    df <- add_row(df, ss)
    progress(i, n_trees, progress.bar = TRUE, init = (i==1)) # print progress
    }

    cat("\nComputing tree summary statistics... Done.")

    names.param <- names(true.param) # names of the parameters (e.g. "lambda" "mu")
    for (name in names.param){
    param.vec  <- true.param[[name]]
    df[name] <- param.vec # store parameter values in data.frame
    }

    return(df)
}
```

Now, if we look at the column names of the dataframe `df` returned by the function `generateSumStatFromPhylo`, we can see that the true values of the parameters are added to the dataframe:

```
> colnames(sumstat)
[1] "a.mean"       "a.med"        "a.var"        "e.mean"       "e.med"
[6] "e.var"        "i.1.mean"     "i.1.med"      "i.1.var"      "i.2.mean"
[11] "i.2.med"      "i.2.var"      "i.3.mean"     "i.3.med"      "i.3.var"
[16] "ie.1.mean"    "ie.1.med"     "ie.1.var"     "ie.2.mean"    "ie.2.med"
[21] "ie.2.var"     "ie.3.mean"    "ie.3.med"     "ie.3.var"     "n_taxa"
```

```
[26] "height"       "colless"      "sackin"       "wdratio"      "deltaw"
[31] "max_ladder"   "il_nodes"     "stair1"       "stair2"       "ltt_slope_1"
[36] "ltt_slope_2"  "ltt_slope_3"  "ltt_slope_4"  "ltt_slope_5"  "ltt_slope_6"
[41] "ltt_slope_7"  "ltt_slope_8"  "ltt_slope_9"  "ltt_slope_10" "ltt_t_1"
[46] "ltt_t_2"      "ltt_t_3"      "ltt_t_4"      "ltt_t_5"      "ltt_t_6"
[51] "ltt_t_7"      "ltt_t_8"      "ltt_t_9"      "ltt_t_10"     "ltt_t_11"
[56] "ltt_t_12"     "ltt_t_13"     "ltt_t_14"     "ltt_t_15"     "ltt_t_16"
[61] "ltt_t_17"     "ltt_t_18"     "ltt_t_19"     "ltt_t_20"     "ltt_N_1"
[66] "ltt_N_2"      "ltt_N_3"      "ltt_N_4"      "ltt_N_5"      "ltt_N_6"
[71] "ltt_N_7"      "ltt_N_8"      "ltt_N_9"      "ltt_N_10"     "ltt_N_11"
[76] "ltt_N_12"     "ltt_N_13"     "ltt_N_14"     "ltt_N_15"     "ltt_N_16"
[81] "ltt_N_17"     "ltt_N_18"     "ltt_N_19"     "ltt_N_20"     "lambda"
[86] "mu"
```

As we can see, the last two columns are the true values of the parameters (in this case of the CRBD model):

```
> head(sumstat$lambda)
[1] 0.8644522 0.3053132 0.5959064 0.9365546 0.5302861 0.6942257

> head(sumstat$mu)
[1] 0.7135123 0.1642631 0.1458384 0.8266150 0.1153805 0.4849967
```

The same is done for the Lineage Through Time encoding, as we can see in the code below:

```
# Lineage Through Time:
ltt <- generate_ltt_dataframe(phylo, taxa_range, true_params)$ltt
saveRDS(ltt, paste("data/phylogeny-", model, "-", expname, "-ltt.rds", sep=""))

    ### generate_ltt_dataframe function - file R/infer-general-functions.R

    generate_ltt_dataframe <- function(trees, n_taxa, true.param){

        n_trees  <- length(trees) # number of trees
```

```
    n_row <- ifelse(length(n_taxa) == 1, n_taxa, n_taxa[2])
    df.ltt <- data.frame("tree1" = rep(NA, n_row))

    df.rates <- as.data.frame(do.call(cbind, true.param))

    cat("Creating LTT dataframe...\n")

    for (i in 1:n_trees){
        tree <- trees[[i]] # get tree
        ltt.coord <- ape::ltt.plot.coords(tree) # get ltt coordinates
        ltt.coord <- as.data.frame(ltt.coord)
        ltt.coord.time <- ltt.coord$time
        n <- length(ltt.coord.time)
        df.ltt[1:n,paste("tree", i, sep = "")] <- ltt.coord$time
        progress(i, n_trees, progress.bar = TRUE, init = (i==1))
    }

    cat("\nCreating LTT dataframe... Done.")

    out <- list("ltt" = df.ltt, "rates" = df.rates) # function output

    return(out)

    }
```

After that, while creating the training and testing sets, the authors used the following code:

```
sumstat <- readRDS(paste("data/phylogeny-", model,
"-", expname, "-sumstat.rds", sep=""))

[...]

train_ds <- ds(sumstat[train_indices, ], names(true), c())
```

We know from the previous code that the dataframe `sumstat` contains the true values of the parameters, so the models were trained with the true values of the parameters inside the training set (after that, no more modifications were made to the training set).

With CBLV encoding the implementation is slighly different, but still not correct:

```
## 02_convert-phylogeny.Rmd

cblv <- generate_encoding(phylo, max_taxa)
saveRDS(cblv, paste("data/phylogeny-", model, "-",
expname, "-cblv.rds", sep=""))

## 03b_predict-cnn-cblv.Rmd

cblv <- readRDS("data/phylogeny-musse-new-sumstat.rds")
true <- readRDS("data/true-parameters-musse-new.rds")
train_ds <- ds(cblv[1:nrow(cblv), train_indices],
               extract_elements(true, train_indices))


    ## extract_elements function - file R/infer-general-functions.R
    ## this function just extracts the true values of the
    parameters from the list of vectors

    extract_elements <- function(list_of_vectors, indices_to_extract){
        l <- as.list(do.call(cbind, list_of_vectors)[indices_to_extract,]
                    %>% as.data.frame())
        return(l)
        }
```

The function `generate_encoding` generates the encoding correctly, but later the true values of the parameters are added to the training set, as we can see in the code above.

For the Phylogeny Graph encoding, the prediction file was empty (`03e_predict-gnn-phy.Rmd`), so I couldn't check if the true values of the parameters were added to the training set, but we can reasonably assume that they were.

This practice is unequivocally prohibited as it fundamentally undermines the integrity of the evaluation process. Consequently, the results obtained under these conditions are rendered entirely invalid and without merit.

4.5. Removing Constraints

The constraints imposed by the authors on the BD and BiSSE model make the model non-fittable to real-world applications: the speciation rate of state 1 is twice the speciation rate of state 0, the extinction rates are fixed to 0 (pure birth model), and the transition rates are symmetric. Since the assumptions are not realistic, I decided to remove the constraints and train the models without them. More specifically, I uniformly sampled every parameter in the given range, without deriving any relation between them, and the same was done for the transition rates, making them unsymmetric. I trained the model (DNN architecture, SS encoding) with the MuSSE model, fine tuning the hyperparameters to get the best results, and the results are shown in Fig. 4.3. This modification was obviosly expected to decrease the accuracy of the predictions, since the model is now more complex and the parameters are not related to each other; but as reported in Fig. 4.3, the predictions are completely off the true values. The distribution of the predictions with respect to the true values let us know that the model is randomly predicting the parameters, understanding only the general range of the parameters. Since the training was done with only 1000 trees, these results are exactly what we would expect: in the previous experiment, MuSSE, with a similar architecture, obtained the same results before I implemented the reordering of the parameters to address the ill-posedness of the problem. So, the results obtained with the MuSSE model are consistent with the previous experiment. This experiment confirms that the constraints imposed by the authors are necessary to obtain good results with the deep learning models, and that the models are not able to learn the relation between the parameters without them, even considering the manipulation of the training set that the authors did - that is, on purpose, still present in this experiment.

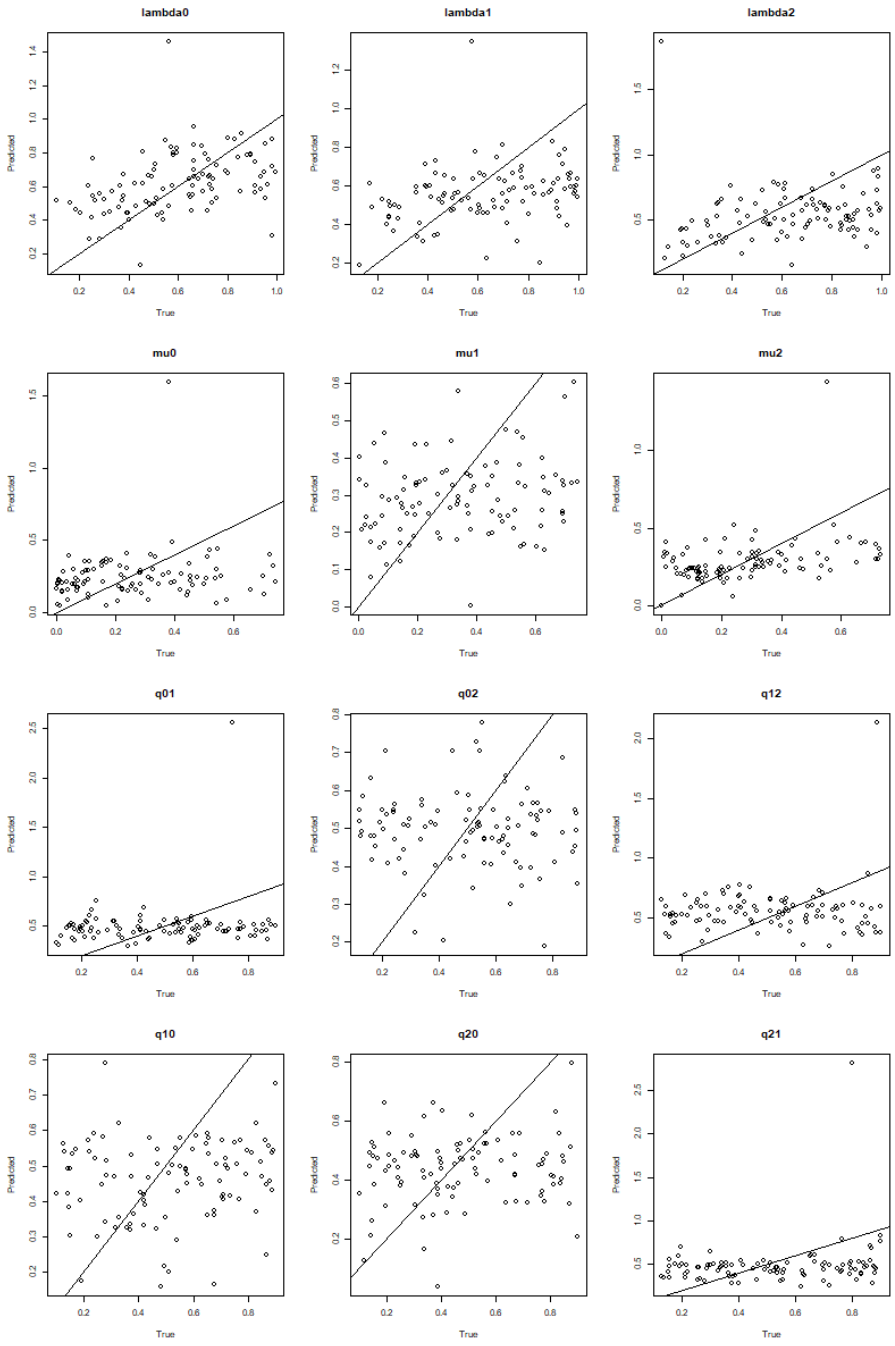


Figure 4.3: MuSSE with 3 states: true vs predicted parameters with DNN architecture and SS encoding, without the constraints imposed by the authors. The black line represents the true values. Even if the model was trained with a manipulated training set, that includes the true values of the parameters, the predictions are completely off the true values, and the model is randomly predicting the parameters: this shows that the strong constraints imposed by the authors play a crucial role in the accuracy of the predictions.

5 | Model Predictor

The model predictor is a new introduction in this field: it is a tool that allows to predict the model that generated a given tree. Without this new approach, the only way to guess it would be to run Maximum Likelihood Estimation for every possible model, with an high probability of not having an analytical solution for some of them.

Various classification models were trained on the trees generated with the different models, and the best model was chosen based on the accuracy of the predictions. I tried both the CDV and SS encoding to compare the results obtained with the different encodings.

The trees were then concatenated, and the targets were stored in a numpy array. The data was then split into training and testing sets, and various classification models were trained on the training set. The accuracy of the predictions was then evaluated on the testing set, and the best model was saved. The models used were a Random Forest Classifier, a Multi-Layer Perceptron Classifier, a Deep Neural Network, and a Convolutional Neural Network. The accuracy of the predictions was evaluated on the testing set, and the best model was saved. The number of available trees, in both cases, was around 30'000, equally distributed among the different models.

The necessary files to reproduce these steps can be found at `https://github.com/Mattia-Colbertaldo/PhyloML`.

5.1. Random Forest Classifier

The Random Forest Classifier is an ensemble learning method that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. Multiple decision trees are trained on different subsets of the data, and the final prediction is made by averaging the predictions of the individual trees. The Random Forest Classifier is a powerful model that can be used for classification tasks, and it is known for its high accuracy and robustness.

In this case, a Random Forest Classifier was trained on the trees generated with the different models. The model was trained with 10000 estimators, and the maximum depth was set to 1000. The model was trained with the trees generated with the different models, and the accuracy of the predictions was evaluated on the testing set.

The Random Forest Classifier also provides a feature importance score, which indicates the importance of each feature in the prediction. The feature importance scores for the CDV encoding are reported in the following table:

```
Feature 1001: 0.0780717705421372
Feature 7: 0.003981114529011713
Feature 4: 0.003974201255265315
Feature 8: 0.003949921477641818
Feature 6: 0.003938771098296724
Feature 10: 0.003924418642942741
Feature 9: 0.003907773523843952
Feature 5: 0.003906910043287 5935
Feature 3: 0.003888391373478 3916
Feature 1: 0.003880836842284 9986
```

The most important feature is feature 1001, which is the last feature in the dataset, representing the rescaling factor.

For the SS encoding, the feature importance scores are reported in Fig. 5.1. The most important feature is feature 97, which is again the last feature in the dataset, still corresponding to the scaling factor.

5.2. Multi-Layer Perceptron Classifier

The Multi-Layer Perceptron Classifier is a type of feedforward neural network that consists of multiple layers of nodes, where each node is connected to all the nodes in the previous layer. In this case, a Multi-Layer Perceptron Classifier was trained on the trees generated with the different models. The model was trained with two hidden layers of 1000 nodes each, and the maximum number of iterations was set to 1000.

5.3. Deep Neural Network

The Deep Neural Network was trained with the following structure:

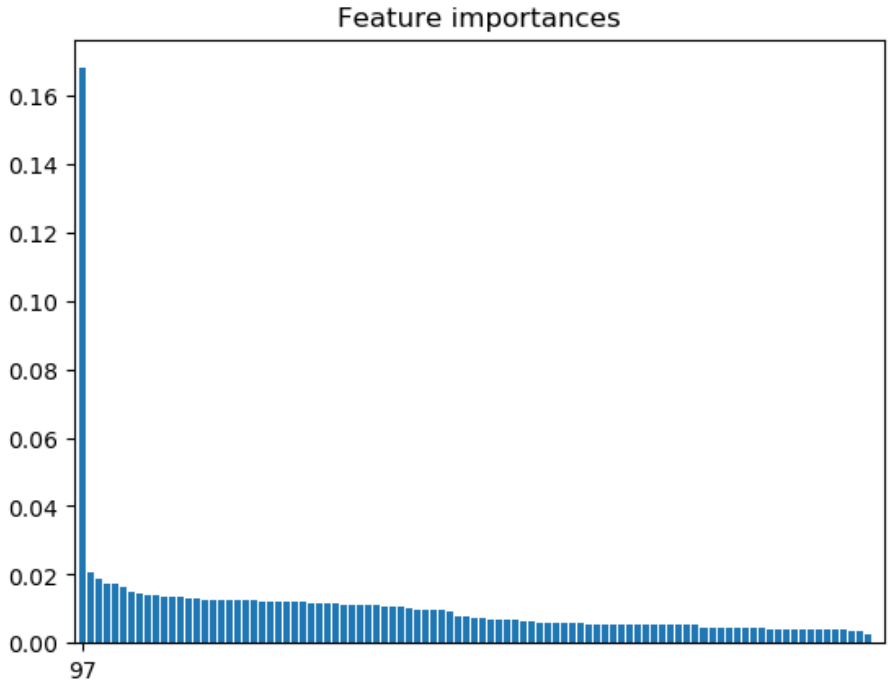


Figure 5.1: Feature importance scores for the Random Forest Classifier trained on the SS encoding. The most important feature is feature 97, which is the last feature in the dataset, and represents the scaling factor.

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 100)               100300
_________________________________________________________________
dense_2 (Dense)              (None, 100)               10100
_________________________________________________________________
dense_3 (Dense)              (None, 100)               10100
_________________________________________________________________
dense_4 (Dense)              (None, 5)                 505
=================================================================
Total params: 120,005
Trainable params: 120,005
Non-trainable params: 0
_________________________________________________________________
```

The model was trained with the Adam optimizer, and the initial learning rate was set to

0.001. The model was trained with the Sparse Categorical Crossentropy loss function. I put the Early Stopping callback, with a patience of 10 epochs, and the model was trained for 100 epochs. The batch size was set to 128.

5.4. Convolutional Neural Network

The Convolutional Neural Network was trained with the following structure:

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
reshape_1 (Reshape)          (None, 1002, 1)           0
_________________________________________________________________
conv1d_3 (Conv1D)            (None, 1000, 32)          128
_________________________________________________________________
max_pooling1d_3 (MaxPooling1 (None, 500, 32)           0
_________________________________________________________________
conv1d_4 (Conv1D)            (None, 498, 64)           6208
_________________________________________________________________
max_pooling1d_4 (MaxPooling1 (None, 249, 64)           0
_________________________________________________________________
conv1d_5 (Conv1D)            (None, 247, 128)          24704
_________________________________________________________________
max_pooling1d_5 (MaxPooling1 (None, 123, 128)          0
_________________________________________________________________
flatten_1 (Flatten)          (None, 15744)             0
_________________________________________________________________
dense_14 (Dense)             (None, 128)               2015360
_________________________________________________________________
dense_15 (Dense)             (None, 5)                 645
=================================================================
Total params: 2,047,045
Trainable params: 2,047,045
Non-trainable params: 0
_________________________________________________________________
```

The model was trained with the Adam optimizer, and the initial learning rate was set to

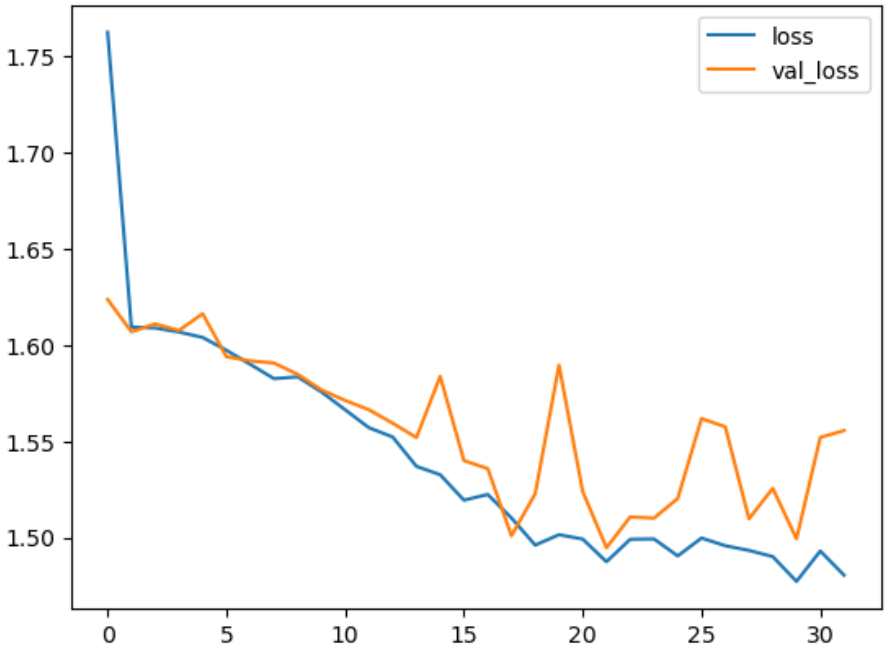


Figure 5.2: History of the training of the Convolutional Neural Network for the model predictor for the CDV encoding.

0.0001. The model was trained with the Sparse Categorical Crossentropy loss function. I put the Early Stopping callback, with a patience of 7 epochs, and the model was trained for 100 epochs. The batch size was set to 128.

The history of the training is shown in Fig. 5.2.

5.5. Conclusions

The performance comparison on the test dataset of the models trained on CDV encoding is shown in Fig. 5.3. The one with the SS encoding is shown in Fig. 5.4. All the models were trained on 10 different seeds, to get the mean accuracy and standard deviation. In both cases the best model is the **Random Forest Classifier**, with a mean accuracy of **62% for the CDV encoding and 68% for the SS encoding**, and an incredibly low standard deviation of 0.0014, that makes the model really reliable: the 95% confidence interval is (0.6786762534601273, 0.6843535612054651)%, which means that, on average, 95% of the times the accuracy of this model is inside that range. The baseline accuracy is 20%, since there are 5 models to choose from, and the accuracy is $1/5 = 20\%$ for a random guess. This new method not only drastically reduces the computational time and resources previously needed to accomplish this task (from minutes to milliseconds) but

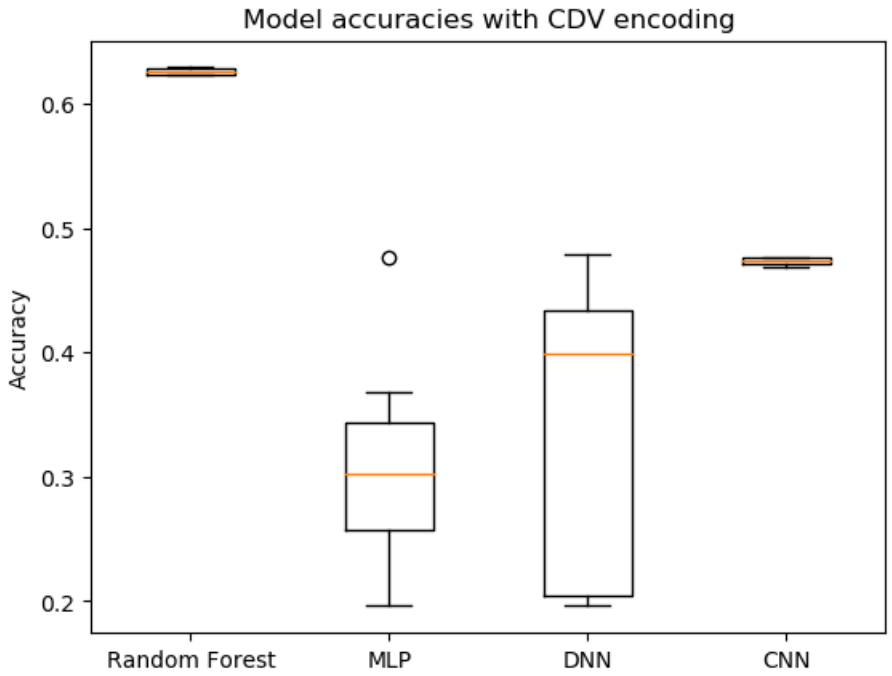


Figure 5.3: Test accuracy of model predictors trained on CDV-encoded trees, on 10 different seeds each. As you can see from the standard deviation of the random forest classifier, its high accuracy is also really reliable.

also makes the inference possible even in cases where it was not, achieving all this with incredibly high accuracy.

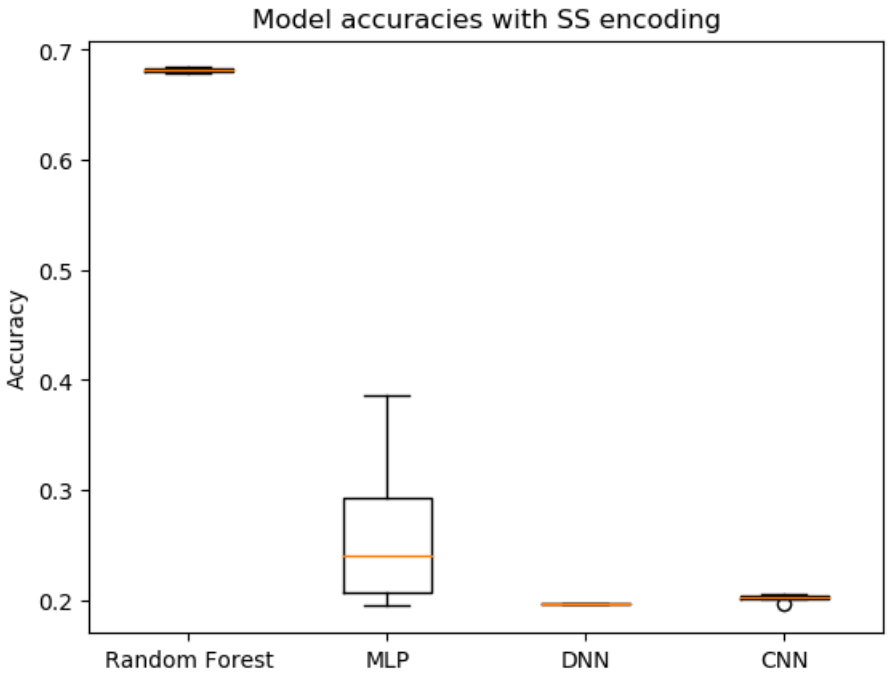


Figure 5.4: Test accuracy of model predictors trained on SS-encoded trees, on 10 different seeds each. As you can see from the standard deviation of the random forest classifier, its high accuracy is also really reliable.

6 | Conclusions and future developments

This work aimed to investigate the use of deep learning techniques for inferring parameters of diversification models from extant phylogenies to substitute Maximum Likelihood Estimation methods, which are slow, computationally expensive, and often lack an analytical solution. The analysis included Convolutional Neural Networks, Linear models, Feed Forward Neural Networks, Gated Recurrent Units (GRU), and Long Short-Term Memory (LSTM) Networks. It led to positive results, only limited by the computational constraints this work was subjected to.

The study mainly involved two types of encoding (CDV and Summary Statistics). Since these encodings do not follow a sequential type of representation, they were not suited for Recurrent Neural Networks (GRU and LSTM), which performed the worst. All the other networks, however, showed good accuracy, even considering the limit on computational resources.

A novel approach was introduced in the field: using a Random Forest Classifier to infer parameters from the trees. Its performance in prediction accuracy was consistently the best compared to other deep learning models, and its training time was a fraction of that of other models.

Additionally, while aiming to extend the work of another research on the matter, it emerged that the declared results were caused by the imposition of strong constraints, which limit the applicability of these models to real-world scenarios.

Furthermore, this work presents a completely new approach in the field: a tool capable of inferring the mathematical model of a given phylogeny. Without it, the only way to guess the diversification model of a phylogeny is to run Maximum Likelihood Estimation

for every possible model, with a high probability of not having an analytical solution for some of them. The study involved the analysis of a Multi-Layer Perceptron, a Linear model, a Convolutional Neural Network, and a Random Forest Classifier. The latter outperformed the others, achieving a really high mean test accuracy of 68%, with a 95% confidence interval of (0.6787, 0.6844)%. This new method not only drastically reduces the computational time and resources previously needed to accomplish this task (from minutes to milliseconds) but also makes the inference possible even in cases where it was not, achieving all this with incredibly high accuracy.

Bibliography

Maddison, W.P., Midford, P.E., Otto, S.P., 2007. Estimating a binary character's effect on speciation and extinction. Syst. Biol. 56, 701–710.

FitzJohn, R.G., Maddison, W.P., Otto, S.P., 2009. Estimating trait-dependent speciation and extinction rates from incompletely resolved phylogenies. Syst. Biol. 58, 595–611.

FitzJohn, R.G., 2010. Quantitative traits and diversification. Syst. Biol. 59, 619–633.

Goldberg, E.E., Lancaster, L.T., Ree, R.H., 2011. Phylogenetic inference of reciprocal effects between geographic range evolution and diversification. Syst. Biol. 60, 451–465.

Magnuson-Ford, K., Otto, S.P., 2012. Linking the investigations of character evolution and species diversification. Am. Nat. 179, 225–245.

Pagel, M., 1994. Detecting correlated evolution on phylogenies: a general method for the comparative analysis of discrete characters. Proc. R. Soc. B Biol. Sci. 255, 37–45.

Lambert S., Voznica J., Morlon H., 2023. Deep Learning from Phylogenies for Diversification Analyses.

FitzJohn, R.G., 2012. Diversitree: comparative phylogenetic analyses of diversification in R. Methods Ecol. Evol. 3, 1084–1092.

A Comparison of Deep Learning Architectures for Inferring Parameters of Diversification Models from Extant Phylogenies Ismaël Lajaaiti, Sophia Lambert, Jakub Voznica, Hélène Morlon, Florian Hartig bioRxiv 2023.03.03.530992; doi: `https://doi.org/10.1101/2023.03.03.530992`

Couvreur, Thomas Kissling, W. Condamine, Fabien Svenning, Jens-Christian Rowe, Nick Baker, William. (2015). Global diversification of a tropical plant growth form: environmental correlates and historical contingencies in climbing palms. Frontiers in Genetics. 5. 452. doi: 10.3389/fgene.2014.00452. 10.3389/fgene.2014.00452.

Johnson, Norman Smith, James Pobiner, Briana Schrein, Caitlin. (2012). Why are Chimps Still Chimps?. The American Biology Teacher. 74. 74-80. 10.1525/abt.2012.74.2.3.

Garcia, M., Theunissen, F., Sèbe, F. *et al.* Evolution of communication signals and information during species radiation. *Nat Commun* 11, 4970 (2020). https://doi.org/10.1038/s41467-020-18772-3

Appendix

.1. Examples

- **Constant BD**: Figure 1

```
set.seed(2)
result <- tree.bd(pars = c(0.5,0.1), max.taxa = 30)
plot(result)
```

- **BiSSE**: Figure 2

```
pars <- c(0.1, 0.2, 0.03, 0.06, 0.01, 0.02)
set.seed(2)
phy <- tree.bisse(pars, max.t=50, x0=0)
col <- c("blue", "orange")
h <- history.from.sim.discrete(phy, 0:1)
plot(h, phy, cex = 0.6, col = col)
```

- **MuSSE**: Figure 3

```
pars <- c(.1, .15, .2, # lambda 1, 2, 3
          .03, .045, .06, # mu 1, 2, 3
          .05, 0, # q12, q13
          .05, .05, # q21, q23
           0, .05) # q31, q32
set.seed(2)
phy <- tree.musse(pars, 30, x0=1)
## Extract history from simulated tree and plot
## (colours are 1: black, 2: red, 3: blue)
col <- c("blue", "orange", "red")
h <- history.from.sim.discrete(phy, 1:3)
plot(h, phy, cex=.7, col=col)
```

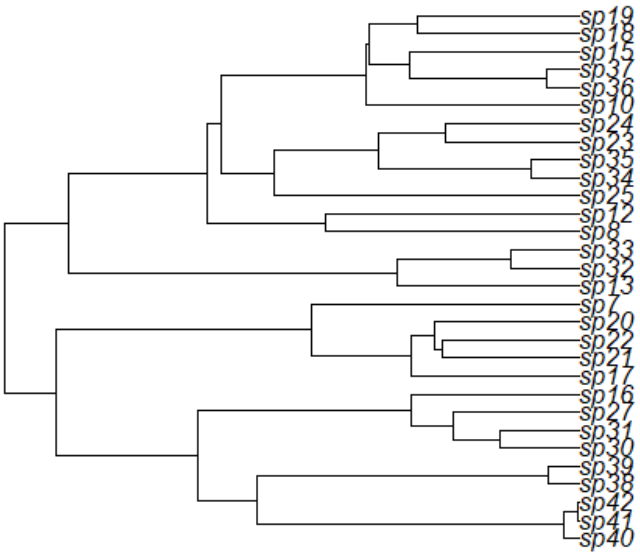


Figure 1: A constant Birth-Death tree, where speciation rate is 0.5 and extinction rate 0.1.

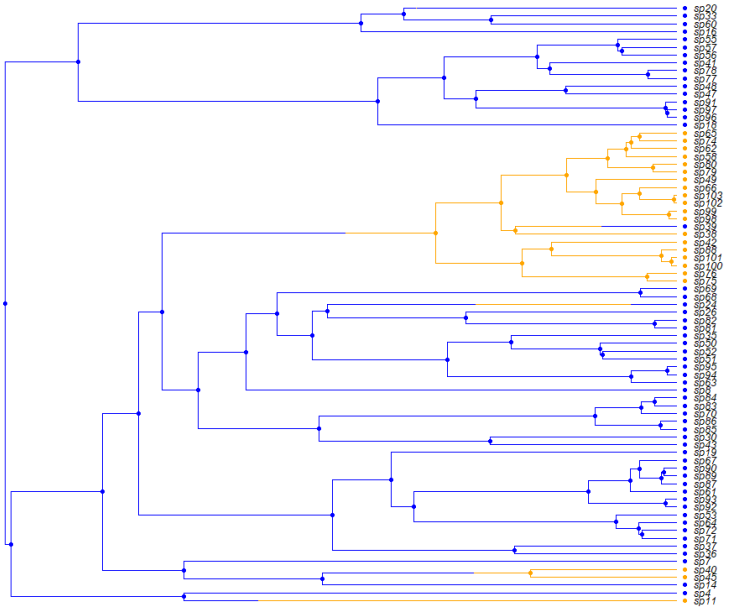


Figure 2: A BiSSE tree, with parameters:
$\lambda_1 = 0.1$, $\lambda_2 = 0.2$,
$\mu_1 = 0.03$, $\mu_2 = 0.06$,
$q_{12} = 0.01$, $q_{21} = 0.02$.
Blue is state 1, orange is state 2.

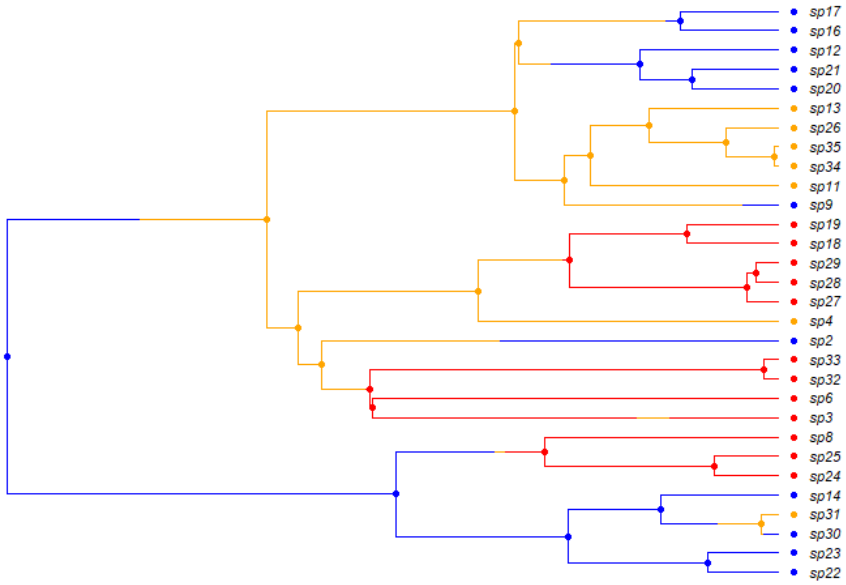


Figure 3: A MuSSE tree, with parameters:
$\lambda_1 = 0.1$, $\lambda_2 = 0.15$, $\lambda_3 = 0.2$,
$\mu_1 = 0.03$, $\mu_2 = 0.045$, $\mu_3 = 0.06$,
$q_{12} = 0.05$, $q_{13} = 0$, $q_{21} = 0.05$, $q_{23} = 0.05$, $q_{31} = 0$, $q_{32} = 0.05$.
Blue is state 1, orange is state 2, red is state 3.

```
## The states are numbered 1:3, rather than 0:1 in bisse.
states <- phy$tip.state
table(states)
```

- **QuaSSE**: Here, speciation rate is a sigmoidal function which ranges from 0.1 to 0.2 with an inflection point at x = 0, extinction is constant at rate 0.03, and the model of character evolution is Brownian motion with diffusion parameter 0.025. Figure 4

```
lambda <- function(x) sigmoid.x(x, 0.1, 0.2, 0, 2.5)
mu <- function(x) constant.x(x, 0.03)
char <- make.brownian.with.drift(0, 0.025)
set.seed(1)
phy <- tree.quasse(c(lambda, mu, char), max.taxa=15, x0=0,
                   single.lineage=FALSE)
plot(phy)
```

- **GeoSSE**: Figure 5

```
pars <- c(1.5, 0.5, 1.0, 0.7, 0.7, 2.5, 0.5)
names(pars) <- diversitree:::default.argnames.geosse()
```

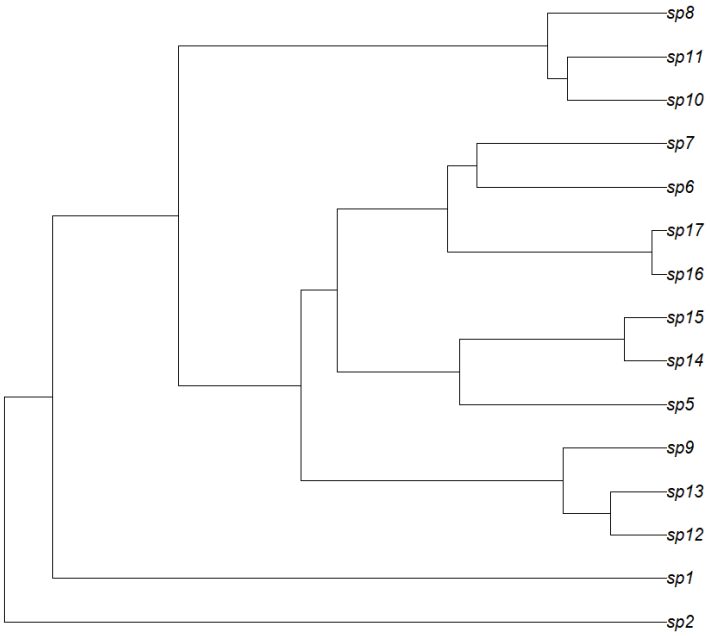


Figure 4: QuaSSE tree, where speciation rate is a sigmoidal function which ranges from 0.1 to 0.2 with an inflection point at x = 0, extinction is constant at rate 0.03, and the model of character evolution is Brownian motion with diffusion parameter 0.025.

```
set.seed(1)
phy <- tree.geosse(pars, max.t=4, x0=0)
statecols <- c("AB"="violet", "A"="blue", "B"="red")
plot(phy, tip.color=statecols[phy$tip.state+1], cex=0.5)
```

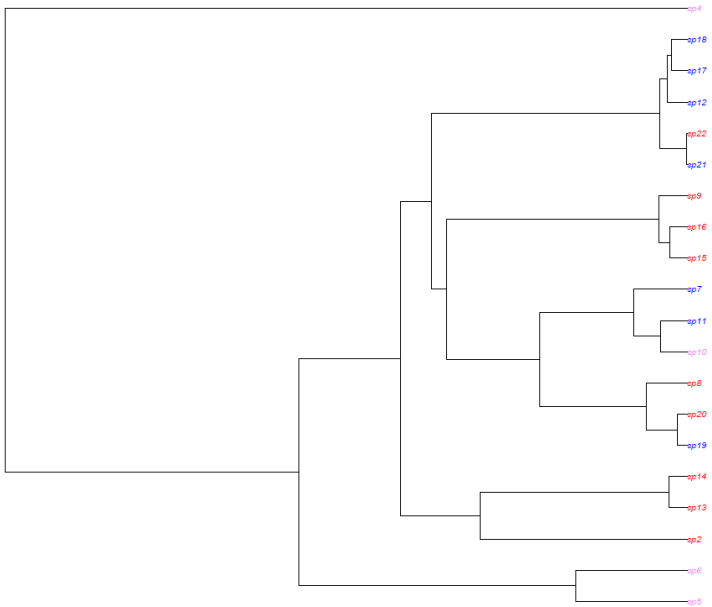


Figure 5: GeoSSE tree, where red represents region A, blue region B and violet the overlap between the two. The parameter values are: $s_A = 1.5$, $s_B = 0.5$, $s_{AB} = 1.0$, $x_A = 0.7$, $x_B = 0.7$, $d_A = 2.5$, $d_B = 0.5$.

- **ClaSSE**: Here we have two character states. Figure 6

```
# diversitree:::default.argnames.classe(2)
#"lambda111" "lambda112" "lambda122" "lambda211" "lambda212" "lambda222" "mu1"
#"mu2"       "q12"       "q21"
pars <- c(0.1, 0.2, 0.03, 0.03, 0.01, 0.01, 0.05, 0.05, 0.1, 0.1)
set.seed(2)
phy <- tree.classe(pars, max.taxa=50, max.t=50, include.extinct=FALSE)
plot(phy)
```

- **BiSSEness**: Figure 7

```
pars <- c(0.1, 0.2, 0.03, 0.06, 0.01, 0.02, 0.1, 0.2, 0.1, 0.2)
```

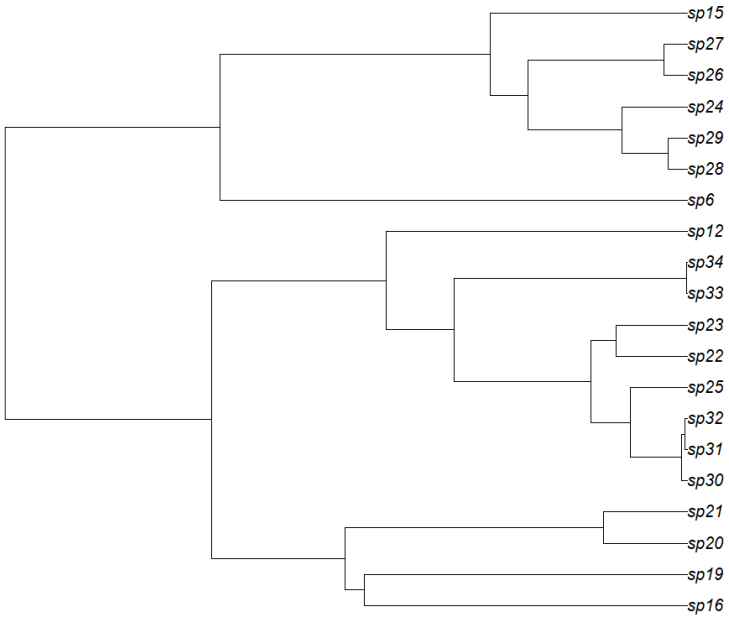


Figure 6: ClaSSE tree, with parameters:
$\lambda_{111} = 0.1$, $\lambda_{112} = 0.2$, $\lambda_{122} = 0.03$,
$\lambda_{211} = 0.03$, $\lambda_{212} = 0.01$, $\lambda_{222} = 0.01$,
$\mu_1 = 0.05$, $\mu_2 = 0.05$,
$q_{12} = 0.1$, $q_{21} = 0.1$.

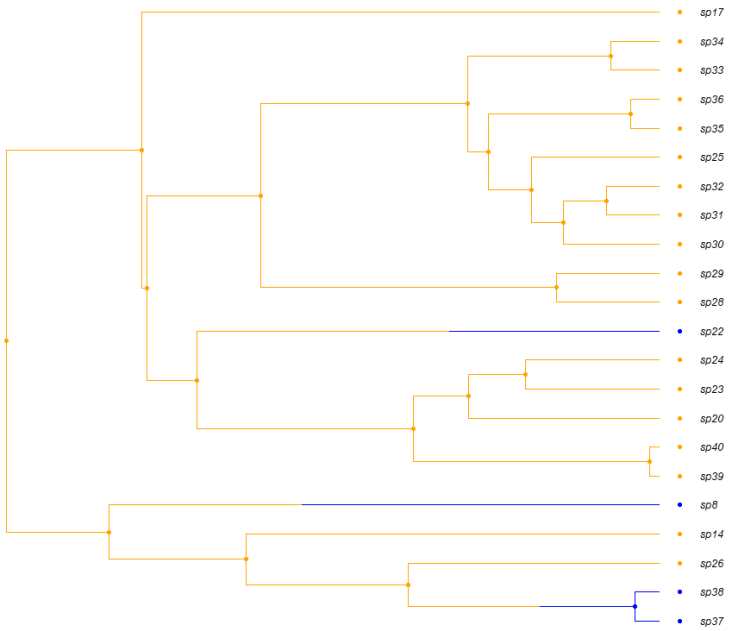


Figure 7: A BiSSEness tree, with parameters:
$\lambda_1 = 0.1$, $\lambda_2 = 0.2$,
$\mu_1 = 0.03$, $\mu_2 = 0.06$,
$q_{12} = 0.01$, $q_{21} = 0.02$,
$p_{0c} = 0.01$, $p_{0a} = 0.02$,
$p_{1c} = 0.01$, $p_{1a} = 0.02$
Blue is state 1, orange is state 2.

```
set.seed(2)
phy <- tree.bisseness(pars, max.t=50, x0=0)
col <- c("blue", "orange")
h <- history.from.sim.discrete(phy, 0:1)
plot(h, phy, cex = 0.6, col = col)
```

.2. Models

- CNN + FFNN:

```
model = Sequential()
model.add(Conv1D(64, 3, activation='linear',kernel_regularizer='l2',
                 input_shape=(features.shape[1], 1)))
model.add(MaxPooling1D(3))
model.add(Conv1D(32, 3, activation='linear',kernel_regularizer='l2'))
model.add(MaxPooling1D(3))
model.add(LSTM(64, return_sequences=True))
model.add(Dropout(0.5))
model.add(LSTM(32))
model.add(Dropout(0.5))
model.add(BatchNormalization())
model.add(Dense(64, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(32, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(16, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(16, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(targets.shape[1], activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(targets.shape[1], activation='linear',kernel_regularizer='l2'))
```

```
- FFNN
```

```
model = Sequential()
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(64, activation="linear", kernel_regularizer='l2'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(32, activation="linear", kernel_regularizer='l2'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(16, activation="linear", kernel_regularizer='l2'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
```

```
model.add(Dense(targets.shape[1], activation="linear", kernel_regularizer='l2'))
```

- GRU

```
model = Sequential()
model.add(layers.GRU(64, return_sequences=True,
                     input_shape=(features.shape[1], 1)))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(layers.GRU(32))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(BatchNormalization())
model.add(Dense(64, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(32, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(16, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(16, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(targets.shape[1], activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.5))
model.add(Dense(targets.shape[1], activation='relu'))
```

- CNN + LSTM

```
model = Sequential()
model.add(Conv1D(64, 3, activation='linear',kernel_regularizer='l2',
                 input_shape=(features.shape[1], 1)))
model.add(MaxPooling1D(3))
```

```
model.add(Conv1D(32, 3, activation='linear',kernel_regularizer='l2'))
model.add(MaxPooling1D(3))
model.add(LSTM(64, return_sequences=True))
model.add(Dropout(0.5))
model.add(LSTM(32))
model.add(Dropout(0.5))
model.add(BatchNormalization())
model.add(Dense(64, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(32, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(16, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(16, activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(targets.shape[1], activation='linear',kernel_regularizer='l2'))
model.add(Dropout(0.5))
model.add(Dense(targets.shape[1], activation='linear',kernel_regularizer='l2'))
```

```
- Dense

model = Sequential()
model.add(Dense(12, input_dim=X.shape[1]))
model.add(Activation('relu'))
```

List of Figures

1.1 Drumming, among woodpeckers species, can be seen as a state differentiation, as it can lead to different speciation and extinction rates. From the figure, it can be seen that non-drummers have a lower speciation and an higer extinction rates, compared to drummers. [12] 4
1.2 BiSSE model: λ represents the speciation rate for each state, while μ the extinction rate. q is the probability of transitioning from one state to another. 6
1.3 Two-states ClaSSE model: global diversification of a tropical plant growth form [10]. 8

2.1 Speciation and extinction rates for real trees. 16
2.2 Trees generated with the constant birth-death model. Red points are null trees, while black points have at least one specie. 17
2.3 Number of species in each tree, generated with the function tree.bd. The number of species is uniformly distributed between 10 and 1000. 19
2.4 Simulation of a phylogenetic tree with the function rlineage, following, in this case, a constant birth-death model. Extinct species are reported. Speciation rate is set to 0.1, while extinction rate is 0.05. 20
2.5 This tree was generated with the function sim.rateshift.taxa, from the package TreeSim. We can see how irregular the pattern is, compared to the constant birth-death models seen before, due to the presence of mass extinction events and rate shifts. 22
2.6 Simulated phylogenetic tree using phytools::pbtree, based on a desired number of extant species (i.e., taxa-stop criterion). Extinct species are omitted. Speciation and extinction rate are constant, respectively 0.1 and 0.05, while the desired number of extant species at present time is set to 20. 23

3.1 Results obtained on BD with the deep learning models trained on the CDV encoding and the SS encoding, and with the MLE method. The plots show the Mean Absolute Error (MAE) for the speciation rate, the extinction rate, net diversification and turnover rate. 26

3.2 Results obtained with the Random Forest Classifier for the BD model. The red scatter points represent the true values, while the blue points represent the predicted values. The points are really close to the line $y = x$, which means that the predictions are accurate. 29
3.3 Predictions obtained with the Random Forest Classifier for the BiSSE model. Predictions are accurate, considering the limitation on the number available trees and the non-uniformly distribution of training data, that lead to notable a bias towards the mean. 30
3.4 Target parameters for the MuSSE model. Note that these values have already been rescaled. As one may expect, due to the non uniform distribution of the parameters, the predictions will be biased towards the mean. Unfortunately, due to the limitation in computational resources, I couldn't cut out any value to make the distribution uniform, since the trees are already too few. 31
3.5 BiSSEness model, one of the most complex to infer. The state-related parameters (from the 5th column on) are difficult to be predicted due to the already discussed reasons. Notably, RNN models (GRU and LSTM, the last two rows) have the worst performance, because Summary Statistics encoding is not sequential. 32
3.6 Boxplot of the error of the predictions obtained with the Random Forest Classifier for the MuSSE model. As said before, the classifier lacks in predicting the transition rates, due to tree structures not containing information about the character states and the limitation of available data , while the speciation and extinction rates are predicted with a good accuracy. 33
3.7 Boxplot of the error of the predictions obtained with the Random Forest Classifier for the GeoSSE model. The classifier performed well in predicting the parameters of the model. GeoSSE is basically the BiSSE model plus a speciation rate among the two geographical regions, so the results were very similar to the ones obtained with the BiSSE model. 34
3.8 Results obtained with the Random Forest Classifier for the ClaSSE model. The red scatter points represent the true values, while the blue points represent the predicted values. Cladogenetic speciation rates and transition rates can't be predicted, due to the fact that tree structures do not contain information about the character states. Extinction rates, instead, are predicted with a good accuracy, as they do not depend on the character states (row 2, columns 2 and 3). 35

4.1 Paper results obtained with the different models and phylogeny representations. 38
4.2 MuSSE with 3 states: True vs predicted parameters with DNN architecture and SS encoding. Only 1000 trees were used to train the models. The black line represents the true values. 41
4.3 MuSSE with 3 states: true vs predicted parameters with DNN architecture and SS encoding, without the constraints imposed by the authors. The black line represents the true values. Even if the model was trained with a manipulated training set, that includes the true values of the parameters, the predictions are completely off the true values, and the model is randomly predicting the parameters: this shows that the strong constraints imposed by the authors play a crucial role in the accuracy of the predictions. 47

5.1 Feature importance scores for the Random Forest Classifier trained on the SS encoding. The most important feature is feature 97, which is the last feature in the dataset, and represents the scaling factor. 51
5.2 History of the training of the Convolutional Neural Network for the model predictor for the CDV encoding. 53
5.3 Test accuracy of model predictors trained on CDV-encoded trees, on 10 different seeds each. As you can see from the standard deviation of the random forest classifier, its high accuracy is also really reliable. 54
5.4 Test accuracy of model predictors trained on SS-encoded trees, on 10 different seeds each. As you can see from the standard deviation of the random forest classifier, its high accuracy is also really reliable. 55

1 A constant Birth-Death tree, where speciation rate is 0.5 and extinction rate 0.1. 62
2 A BiSSE tree, with parameters: $\lambda_1 = 0.1$, $\lambda_2 = 0.2$, $\mu_1 = 0.03$, $\mu_2 = 0.06$, $q_{12} = 0.01$, $q_{21} = 0.02$. Blue is state 1, orange is state 2. 63
3 A MuSSE tree, with parameters: $\lambda_1 = 0.1$, $\lambda_2 = 0.15$, $\lambda_3 = 0.2$, $\mu_1 = 0.03$, $\mu_2 = 0.045$, $\mu_3 = 0.06$, $q_{12} = 0.05$, $q_{13} = 0$, $q_{21} = 0.05$, $q_{23} = 0.05$, $q_{31} = 0$, $q_{32} = 0.05$. Blue is state 1, orange is state 2, red is state 3. 64
4 QuaSSE tree, where speciation rate is a sigmoidal function which ranges from 0.1 to 0.2 with an inflection point at x = 0, extinction is constant at rate 0.03, and the model of character evolution is Brownian motion with diffusion parameter 0.025. 65

5 GeoSSE tree, where red represents region A, blue region B and violet the overlap between the two. The parameter values are: $s_A = 1.5$, $s_B = 0.5$, $s_{AB} = 1.0$, $x_A = 0.7$, $x_B = 0.7$, $d_A = 2.5$, $d_B = 0.5$. 66
6 ClaSSE tree, with parameters: $\lambda_{111} = 0.1$, $\lambda_{112} = 0.2$, $\lambda_{122} = 0.03$, $\lambda_{211} = 0.03$, $\lambda_{212} = 0.01$, $\lambda_{222} = 0.01$, $\mu_1 = 0.05$, $\mu_2 = 0.05$, $q_{12} = 0.1$, $q_{21} = 0.1$. . 67
7 A BiSSEness tree, with parameters: $\lambda_1 = 0.1$, $\lambda_2 = 0.2$, $\mu_1 = 0.03$, $\mu_2 = 0.06$, $q_{12} = 0.01$, $q_{21} = 0.02$, $p_{0c} = 0.01$, $p_{0a} = 0.02$, $p_{1c} = 0.01$, $p_{1a} = 0.02$ Blue is state 1, orange is state 2. 68

List of Tables

Model	Parameters	Description
const BD	λ, μ	Models pure birth-death processes without character state transitions.
BiSSE	$\lambda_1, \lambda_2, \mu_1, \mu_2, q_{12}, q_{21}$	Constant rates of speciation, extinction, and transition between two character states.
MuSSE	$\lambda_1, \ldots, \lambda_K, \mu_1, \ldots, \mu_K, q_{ij}$	Generalizing BiSSE to K character states with constant rates.
QuaSSE	Arbitrary functions of trait values	Allows for variation of speciation and extinction rates as functions of trait values.
GeoSSE	$s_A, s_B, s_{AB}, x_A, x_B, d_A, d_B$	Accounts for geographical considerations with constant rates of speciation, extinction, and dispersal.
ClaSSE	λ_{klm} (where $1 \leq k, l, m \leq n$), μ_k (where $1 \leq k \leq n$), q_{lm} (where $1 \leq l \neq m \leq n$)	Distinguishes between anagenetic and cladogenetic speciation with variable rates. There will be n^3 cladogenesis rates and n extinction rates plus $n(n-1)$ transition rates, making a total of $n^3 + n + n(n-1)$ parameters.
BiSSEness	$\lambda_0, \lambda_1, \mu_0, \mu_1, q_{12}, q_{21}$, $p_{0c}, p_{0a}, p_{1c}, p_{1a}$	Estimates rates of change occurring along lineages and probabilities of change during speciation.

List of Symbols

Variable	Description
λ_i	speciation rate of state i
μ_i	extinction rate of state i
q_{ij}	transition rate from state i to state j

Acknowledgements

To my beloved family, Valentina, and my close friends.